

System Design Interviews

Top 20 Questions — 7-Step Framework + AWS Well-Architected Review

Functional & Non-Functional Requirements · Core Entities · API · High-Level Design · Deep Dives · Trade-offs · 6
AWS Pillars

Table of Contents

- System Design Interviews — Top 20 Questions
- Part A — Questions 1–5
 - 1. URL Shortener (TinyURL / Bitly)
 - 2. Pastebin
 - 3. Twitter / News Feed
 - 4. Instagram
 - 5. WhatsApp / Chat System
- Part B — Questions 6–10
 - 6. Rate Limiter
 - 7. YouTube / Netflix (Video Streaming)
 - 8. Web Crawler
 - 9. Notification System
 - 10. Uber / Ride-Sharing
- Part C — Questions 11–15
 - 11. Google Drive / Dropbox (File Storage & Sync)
 - 12. Ticketmaster (Ticket Booking)
 - 13. Distributed Cache
 - 14. Search Autocomplete / Typeahead
 - 15. Google Maps (Geospatial + Routing)
- Part D — Questions 16–20
 - 16. Distributed Message Queue (Kafka-like)
 - 17. Leaderboard / Ranking System
 - 18. Payment System / Digital Wallet
 - 19. Key-Value Store (DynamoDB / Cassandra-like)
 - 20. DoorDash / Food Delivery

System Design Interviews — Top 20 Questions

A structured, interview-ready workbook. Every design follows the **same 7-step framework** and closes with an **AWS Well-Architected Framework** assessment across all **6 pillars**.

The 7-Step Design Framework (and *why* each step exists)

Interviewers care less about the "right answer" and more about a **structured, requirements-driven thought process**. This framework keeps you organized and demonstrates senior-level judgment. Follow it in order for every problem.

Step 1 — Functional Requirements

What: The concrete features the system must provide ("users can shorten a URL and be redirected").

Why: Scopes the problem. Interview time is limited, so you agree on the *core* features to design (and explicitly defer the rest). Prevents building the wrong thing. Always state what's **in scope** and **out of scope**.

Step 2 — Non-Functional Requirements (NFRs)

What: The qualities the system must have — scale (users/QPS), latency, availability, consistency, durability, read/write ratio.

Why: NFRs *drive the architecture* far more than features do. "1K users" and "100M users" have identical features but completely different designs. This is where you do **back-of-the-envelope estimation** (QPS, storage, bandwidth) to size the system.

Step 3 — Core Entities

What: The main data objects and their relationships (User, URL, Tweet, Order...).

Why: Establishes the vocabulary and the data model foundation. It bridges requirements and API/schema, and surfaces relationships that later inform the database choice and partitioning.

Step 4 — The API

What: The contract between clients and the system — endpoints, methods, inputs, outputs.

Why: Defines the system's boundary and how each functional requirement is fulfilled. Designing the API early forces clarity on exactly what the system does, and naturally reveals read vs write paths (which you then scale differently).

Step 5 — High-Level Design

What: The major components (clients, load balancer, services, databases, caches, queues) and how data flows between them, usually as a diagram.

Why: This is the heart of the interview — the architecture that satisfies the requirements. Start simple (the minimum that works), then evolve. Show the request path for each core API.

Step 6 — Potential Deep Dives

What: Focused exploration of the hardest/most interesting parts (the bottleneck, the hot partition, the consistency challenge, the scaling limit).

Why: Demonstrates depth. Interviewers probe here to separate memorized answers from real understanding. This is where you handle scale, edge cases, and failure modes.

Step 7 — Trade-offs

What: The conscious choices you made and their alternatives (SQL vs NoSQL, strong vs eventual consistency, sync vs async).

Why: Senior engineering *is* trade-off analysis. There's no perfect design — showing you understand what you gained and gave up, and *why it fits the requirements*, is the strongest signal you can send.

AWS Well-Architected Framework — the 6 Pillars

Each design closes with a Well-Architected review. The framework is AWS's set of best practices for building secure, reliable, efficient, cost-effective, and sustainable systems. The **6 pillars**:

1. **Operational Excellence** — run and monitor systems to deliver value; automate, use infrastructure as code, make small reversible changes, observe everything, and learn from failures.
2. **Security** — protect data and systems: strong identity/least-privilege access, encryption in transit and at rest, detective controls, and incident response.
3. **Reliability** — recover from failures and meet demand: no single points of failure, automated recovery, horizontal scaling, and defined RPO/RTO.
4. **Performance Efficiency** — use resources efficiently as demand evolves: right-size, pick the right services, cache, and monitor.
5. **Cost Optimization** — avoid unnecessary spend: match supply to demand, use appropriate pricing models, and measure cost against value.
6. **Sustainability** — minimize environmental impact: efficient resource use, right-sizing, serverless/managed services, and efficient regions.

Why assess against it: It turns "looks good" into a rigorous, multi-dimensional review, and forces explicit trade-offs between pillars (e.g., higher reliability often costs more) — exactly the judgment senior roles demand.

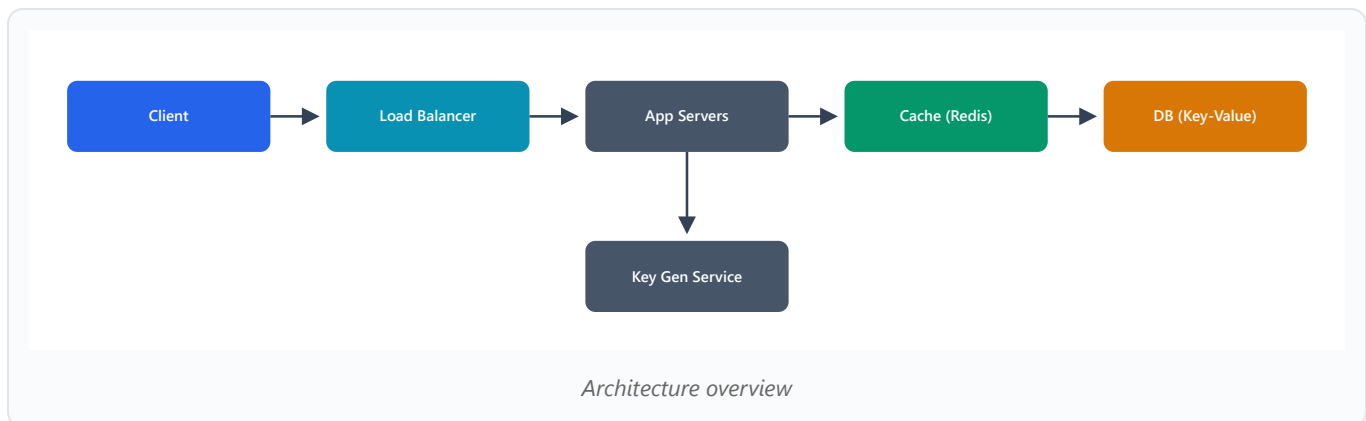
The 20 Questions

#	System	Primary challenge it teaches
1	URL Shortener (TinyURL)	Key generation, read-heavy scaling, caching
2	Pastebin	Blob storage, expiry, read scaling
3	Twitter / News Feed	Fan-out, feed generation, the celebrity problem
4	Instagram	Media storage, feed, CDN
5	WhatsApp / Chat	Real-time messaging, presence, delivery guarantees
6	Rate Limiter	Algorithms, distributed counters, low latency
7	YouTube / Netflix	Video upload, transcoding, streaming, CDN
8	Web Crawler	Large-scale crawling, politeness, dedup
9	Notification System	Multi-channel fan-out, reliability, dedup
10	Uber / Ride-Sharing	Geospatial matching, real-time location
11	Google Drive / Dropbox	File sync, chunking, conflict resolution
12	Ticketmaster	Reservations, concurrency, preventing double-booking
13	Distributed Cache	Consistent hashing, eviction, replication
14	Search Autocomplete	Trie, prefix queries, ranking, low latency
15	Google Maps	Geospatial data, routing, tiles
16	Distributed Message Queue	Durability, ordering, delivery semantics
17	Leaderboard	Sorted sets, real-time ranking at scale
18	Payment System	Idempotency, consistency, exactly-once, audit
19	Key-Value Store	Partitioning, replication, quorums (Dynamo)
20	DoorDash / Food Delivery	Three-sided marketplace, geospatial, logistics

Each write-up is self-contained. Read any in isolation, or all in sequence to build broad coverage.

Part A — Questions 1–5

1. URL Shortener (TinyURL / Bitly)



Problem: Design a service that converts a long URL into a short one and redirects users from the short URL to the original.

1. Functional Requirements

- **Shorten:** given a long URL, return a unique short URL.
- **Redirect:** given a short URL, redirect (HTTP 302) to the original long URL.
- **Custom alias (optional):** let users pick a custom short code.
- **Expiration (optional):** links can expire at a chosen time.
- *Out of scope:* user accounts, analytics dashboards (mention them, defer them).

2. Non-Functional Requirements

- **Read-heavy:** redirects vastly outnumber creations — assume **100:1 read:write**.
- **Scale:** ~100M new URLs/month → ~40 writes/sec; ~4,000 redirects/sec; 5B redirects/month.
- **Low latency:** redirects must feel instant (< 100 ms P99).
- **High availability:** a broken redirect is a broken link — target 99.99%.
- **Durability:** links must not be lost.
- *Estimation:* 100M/month × 12 × 5 years ≈ **6B URLs**; ~500 bytes each ≈ **3 TB** — a single well-provisioned DB can hold this, but we plan for growth.

3. Core Entities

- **URL mapping:** `shortCode` (PK), `longUrl`, `createdAt`, `expiresAt`, `ownerId?`.
- **User (optional):** `userId`, for custom aliases/ownership.

4. The API

```
POST /urls          → create short URL
body: { longUrl, customAlias?, expiresAt? }
```

```
returns: { shortUrl }
```

```
GET /{shortCode} → 302 redirect to longUrl
```

The write path (`POST`) is low volume; the read path (`GET /{shortCode}`) is the hot path we scale hardest.

5. High-Level Design

```
Client → Load Balancer → App servers → Cache (Redis) → Database (KV / SQL)
                        |
                        Key Generation Service
```

- **Create:** app generates a unique `shortCode`, stores `shortCode → longUrl` in the DB, returns the short URL.
- **Redirect:** app looks up `shortCode` → check cache first → on miss, read DB, populate cache → return 302.
- **The key question — how to generate short codes:** encode a unique number in **base62** (`[a-zA-Z0-9]`). $62^7 \approx 3.5$ trillion codes in 7 chars — plenty. Get the unique number from either a distributed counter/ID generator or a pre-generated key range per server.

6. Potential Deep Dives

- **Key generation without collisions:**
- *Counter + base62:* a global counter (or ranges handed to each server via ZooKeeper/DB) guarantees uniqueness with no collision checks. Simple and scalable.
- *Hashing (MD5/SHA of URL, take 7 chars):* risks collisions → need a collision check/retry. Also identical URLs collapse (may or may not be desired).
- **Choice:** counter-based ranges — collision-free and fast.
- **Read scaling:** the redirect path is read-heavy → **cache aggressively** (Redis, LRU). Hot links live in cache; hit rate is very high because access is skewed. Add read replicas and a CDN for the redirect endpoint.
- **Database choice:** a simple key-value lookup by `shortCode` → a KV store (DynamoDB/Cassandra) scales horizontally and is a natural fit; a single SQL DB also works at this scale. Partition by `shortCode`.
- **Custom aliases:** check uniqueness on write (unique constraint / conditional put); reject duplicates.
- **Expiration & cleanup:** store `expiresAt`; a background job (or TTL in the KV store) purges expired links.

7. Trade-offs

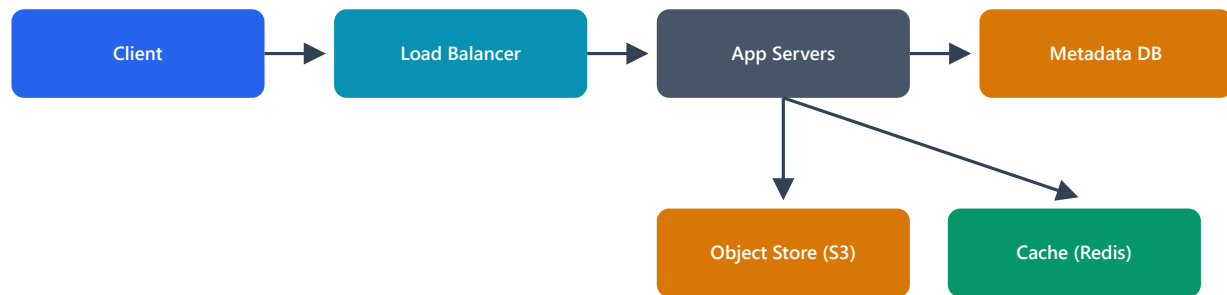
- **302 vs 301 redirect:** 302 (temporary) lets us keep serving every request (enabling analytics and expiry control) at the cost of not being cached by browsers; 301 (permanent) is cached by browsers (less load) but you lose per-click visibility. Choose **302** for control.

- **Counter vs hashing:** counter is collision-free but needs coordination; hashing is stateless but risks collisions. We chose coordination for correctness.
- **KV vs SQL:** KV scales writes/storage effortlessly but no rich queries; SQL is simpler and fine at this scale. We lean KV for headroom.

AWS Well-Architected Assessment

- **Operational Excellence:** Managed services (DynamoDB, ElastiCache, ALB) reduce ops; IaC (CloudFormation/CDK) for reproducibility; CloudWatch dashboards on redirect latency and error rate.
 - **Security:** WAF on the edge to block malicious/abusive shorten requests; validate/sanitize input URLs (prevent open-redirect abuse to phishing); TLS everywhere; least-privilege IAM for services.
 - **Reliability:** DynamoDB multi-AZ + global tables; ElastiCache with replicas; the redirect path degrades gracefully to DB if cache is down. No SPOF.
 - **Performance Efficiency:** CloudFront (CDN) + Redis cache serve most redirects; DynamoDB on-demand scales with traffic; base62 keeps codes compact.
 - **Cost Optimization:** DynamoDB on-demand/auto-scaling matches the spiky read load; caching cuts DB read cost dramatically (reads are the volume driver).
 - **Sustainability:** Serverless/managed (DynamoDB, Lambda/Fargate) right-sizes automatically; high cache hit rate means fewer compute/DB cycles per request.
-

2. Pastebin



Architecture overview

Problem: Design a service where users paste text (or code) and get a shareable link to view it later.

1. Functional Requirements

- **Create paste:** submit text, get a unique URL.
- **Read paste:** view the paste via its URL.
- **Expiration:** pastes can expire (time-based) or be one-time-view.
- **Optional:** custom URL, visibility (public/unlisted).
- *Out of scope:* editing, comments, syntax highlighting (client concern).

2. Non-Functional Requirements

- **Read-heavy** (a paste is written once, read many times).
- **Scale:** ~10M new pastes/month; reads 10× writes.
- **Size:** pastes up to a few MB → text is a **blob**, not a DB row field.
- **Low read latency**, high availability, durable storage.
- *Estimation:* average paste ~10 KB; 10M/month × 10 KB ≈ **100 GB/month** → object storage, not a database.

3. Core Entities

- **Paste:** `pasteId` (PK), `contentPointer` (object-store key), `createdAt`, `expiresAt`, `visibility`, `viewOnce?`.
- **User (optional).**

4. The API

```
POST /pastes          → { content, expiresAt?, visibility? } → { pasteId, url }
GET /pastes/{pasteId} → returns the paste content (metadata + text)
```

5. High-Level Design

```
Client → LB → App servers → Metadata DB (pasteId → object key, expiry)
                        |
                        |→ Object Storage (S3) for the actual text blob
                        |→ Cache (Redis) for hot pastes
```

- **Create:** generate `pasteId` (base62 of a unique id), write the **text to object storage** (key = `pasteId`), write **metadata to the DB**, return URL.
- **Read:** look up metadata (DB/cache) → fetch blob from object storage (or cache) → return.
- **Why split blob vs metadata:** databases are bad at large blobs; object storage is cheap, durable, and CDN-friendly. The DB holds only small, queryable metadata.

6. Potential Deep Dives

- **Storage separation:** metadata in a small fast store (DynamoDB/SQL), content in S3. Cache small hot pastes in Redis; serve large/popular ones via CDN.
- **Expiration:** store `expiresAt`; use S3 lifecycle rules + a metadata TTL/cleanup job. For **one-time view**, mark as read (atomic update) then delete/soft-delete on first fetch.
- **ID generation:** same base62 counter approach as the URL shortener.
- **Read scaling:** CDN in front of the read endpoint for public pastes; cache metadata to avoid DB hits.
- **Abuse/limits:** cap paste size; rate-limit creation; scan for malware/secrets if needed.

7. Trade-offs

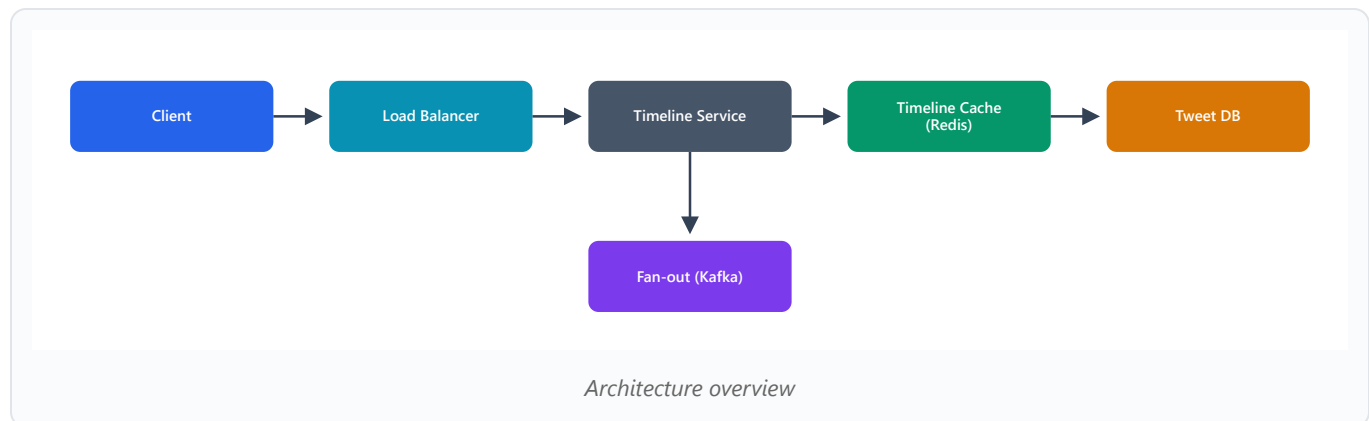
- **Object storage vs DB blob:** object storage wins on cost, durability, and size limits; the trade-off is a second lookup (metadata → blob), mitigated by caching.
- **One-time view consistency:** enforcing "exactly one view" needs an atomic check — a small consistency cost on read.
- **Public caching vs privacy:** CDN caching boosts performance but is only safe for public/unlisted pastes; private pastes bypass the CDN.

AWS Well-Architected Assessment

- **Operational Excellence:** S3 lifecycle policies automate expiry/cleanup; CloudWatch metrics on create/read rates; IaC for the stack.
- **Security:** encrypt content at rest (S3 SSE) and in transit; signed URLs for private pastes; input size limits and WAF to curb abuse; least-privilege access to the bucket.
- **Reliability:** S3 gives 11-nines durability and multi-AZ; metadata DB multi-AZ; system tolerates cache loss.

- **Performance Efficiency:** CloudFront for popular pastes; Redis for metadata; S3 handles blob scale effortlessly.
 - **Cost Optimization:** S3 (cheap storage) + lifecycle tiering (move cold pastes to IA/Glacier) + TTL cleanup avoid paying for dead data; serverless compute scales to zero.
 - **Sustainability:** Tiering and auto-expiry delete unused data (less stored bytes); managed services right-size compute.
-

3. Twitter / News Feed



Problem: Design the core of Twitter — post tweets, follow users, and see a timeline of tweets from people you follow.

1. Functional Requirements

- **Post a tweet** (text, up to 280 chars, optional media).
- **Follow/unfollow** users.
- **Home timeline:** see recent tweets from followed users, newest first.
- **User timeline:** see a specific user's own tweets.
- *Out of scope (mention/defer):* likes, retweets, search, DMs, trends.

2. Non-Functional Requirements

- **Extremely read-heavy** (timeline reads $\gg$ tweet writes).
- **Scale:** ~300M DAU; ~500M tweets/day (~6K writes/sec, higher peaks); timeline reads ~10 $\times$ or more.
- **Low latency:** timeline loads < 200 ms.
- **High availability** > consistency: eventual consistency is fine (a tweet appearing a second late is acceptable).
- *Estimation:* 500M tweets/day $\times$ 300 bytes $\approx$ **150 GB/day** of text; media dwarfs this and goes to object storage + CDN.

3. Core Entities

- **User:** `userId`, `handle`, `profile`.
- **Tweet:** `tweetId`, `authorId`, `text`, `mediaUrl?`, `createdAt`.
- **Follow:** `followerId` $\rightarrow$ `followeeId` (the social graph).
- **Timeline:** a per-user, precomputed list of tweetIds (in the fan-out-on-write model).

4. The API

POST /tweets	→ { text, media? } → { tweetId }
POST /users/{id}/follow	→ follow a user
GET /feed?cursor=&limit=	→ home timeline (paginated)
GET /users/{id}/tweets?cursor=	→ a user's tweets

5. High-Level Design

```

Write:  Client → LB → Tweet Service → Tweet DB
        |
        └→ Fan-out Service → Timeline Cache (per-user lists in Redis)

Read:   Client → LB → Timeline Service → Timeline Cache (Redis) → hydrate from Tweet DB

```

- **The central problem — how to build the home timeline** for 300M users cheaply and fast. Two strategies:
- **Fan-out on write (push):** when a user tweets, push the tweetId into every follower's precomputed timeline list. **Reads are O(1)** (just read your list) — great for the read-heavy workload.
- **Fan-out on read (pull):** build the timeline on demand by querying all followees' recent tweets and merging. Cheap writes, expensive reads.
- **Chosen approach: hybrid** — fan-out on write for most users; fan-out on read for celebrities (see deep dive).

6. Potential Deep Dives

- **The celebrity problem (hot key):** fan-out on write breaks for a user with 100M followers — one tweet = 100M timeline writes (a write storm). **Solution:** don't fan out celebrity tweets; instead, at **read** time, merge the (few) celebrities you follow into your precomputed timeline. This **hybrid** bounds write amplification while keeping normal reads O(1).
- **Timeline storage:** store each user's timeline as a capped list (e.g., latest 800 tweetIds) in Redis; hydrate tweet content from the Tweet DB/cache at read time (store IDs, not full tweets, to save memory).
- **Tweet storage & sharding:** shard tweets by `tweetId` (or `authorId`); use a KV/wide-column store (Cassandra) for write throughput. Media → object storage + CDN.
- **Feed ranking:** chronological is simplest; ranked feeds add an ML scoring step over candidate tweets.
- **Consistency:** eventual — acceptable for feeds. Use async fan-out via a queue (Kafka) to absorb bursts and decouple posting from delivery.

7. Trade-offs

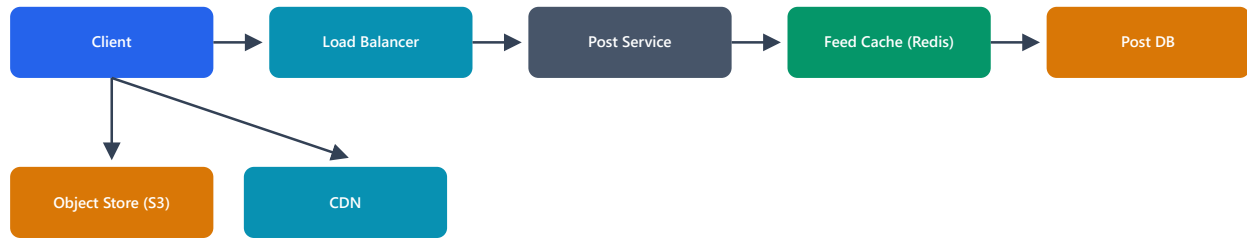
- **Push vs pull:** push = fast reads, costly writes (and the celebrity storm); pull = cheap writes, slow reads. Hybrid gets the best of both at the cost of complexity.
- **Store IDs vs full tweets in timelines:** IDs save memory but need a hydration step; full copies are faster to read but expensive and stale-prone. Choose IDs + cache.

- **Consistency vs availability:** we accept eventual consistency to get high availability and low latency — correct for social feeds.

AWS Well-Architected Assessment

- **Operational Excellence:** Kafka (MSK) decouples fan-out; CloudWatch/X-Ray trace the write→fan-out→read path; canary deploys for ranking changes.
 - **Security:** authN/authZ on all endpoints; rate limiting to stop spam/bots; content moderation hooks; encryption in transit/at rest.
 - **Reliability:** async fan-out with a durable queue survives spikes/consumer failures; multi-AZ Cassandra/Redis; degraded mode (pull) if the timeline cache fails.
 - **Performance Efficiency:** precomputed timelines make reads $O(1)$; Redis + CDN for media; hybrid fan-out avoids the celebrity bottleneck.
 - **Cost Optimization:** storing IDs (not full tweets) in Redis slashes memory cost; async batching of fan-out smooths capacity; tiered storage for old tweets.
 - **Sustainability:** avoiding celebrity write-storms (hybrid) saves enormous compute; capped timelines and cold-tiering reduce stored data.
-

4. Instagram



Architecture overview

Problem: Design a photo/video sharing app — upload media, follow users, and view a feed.

1. Functional Requirements

- **Upload** photos/videos with captions.
- **Follow** users.
- **Feed:** view recent posts from followed users.
- **View a user's profile/posts.**
- *Out of scope:* stories, DMs, explore/search (mention, defer).

2. Non-Functional Requirements

- **Read-heavy;** media-heavy (large payloads).
- **Scale:** ~500M DAU; ~100M posts/day; billions of feed views/day.
- **Low latency** for feed and media load; **high availability;** eventual consistency OK.
- *Estimation:* 100M posts/day × ~2 MB avg media ≈ **200 TB/day** of media → object storage + CDN are mandatory.

3. Core Entities

- **User, Post** (`postId` , `authorId` , `mediaUrls` , `caption` , `createdAt`), **Follow, Feed** (per-user list of `postIds`).

4. The API

```
POST /media/upload-url → returns a pre-signed upload URL (client uploads directly to storage)
POST /posts           → { mediaKeys, caption } → { postId }
GET /feed?cursor=&limit= → feed of posts
GET /users/{id}/posts  → a user's posts
```

5. High-Level Design

```
Upload: Client → (pre-signed URL) → Object Storage (S3) directly
        Client → Post Service → Post DB → Fan-out → Feed Cache (Redis)
Media:   Object Storage → CDN → Client
Read:    Client → Feed Service → Feed Cache → hydrate Post DB → media via CDN
```

- **Direct-to-storage upload:** the client requests a **pre-signed URL** and uploads the media *directly* to S3, bypassing app servers (huge bandwidth/scaling win). The app only stores metadata.
- **Feed:** same fan-out model as Twitter (hybrid push/pull).
- **Media delivery:** always via **CDN** — this is the dominant traffic and latency factor.

6. Potential Deep Dives

- **Media pipeline:** on upload, trigger async processing (generate multiple resolutions/thumbnails, transcode video) via a queue + workers; store variants in S3; serve the right size per device via CDN.
- **Feed generation:** fan-out-on-write with the celebrity exception, identical reasoning to Twitter.
- **CDN & caching:** aggressive CDN caching of immutable media (content-hashed keys); cache feed lists in Redis.
- **Storage tiering:** old media → cheaper storage tiers; hot media stays on CDN edges.
- **Consistency:** eventual for feed; media becomes available after processing completes (show a spinner/placeholder meanwhile).

7. Trade-offs

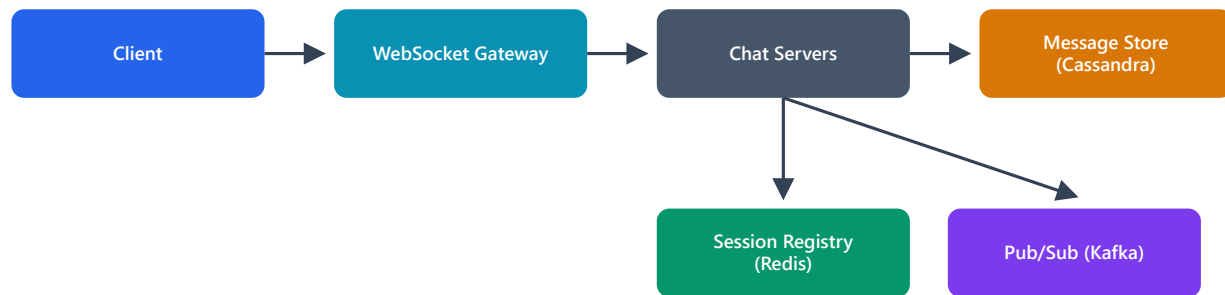
- **Pre-signed direct upload vs proxy upload:** direct upload offloads bandwidth and scales better but exposes storage details (mitigated by scoped, short-lived URLs).
- **Multiple resolutions (storage) vs on-the-fly resize (compute):** pre-generating variants costs storage but gives fast, cacheable delivery; on-the-fly saves storage but adds latency/compute. Pre-generate for the read-heavy case.
- **Eventual consistency:** feed and media availability lag slightly — acceptable for UX.

AWS Well-Architected Assessment

- **Operational Excellence:** S3 event → Lambda/Fargate transcoding pipeline is fully automated; CloudWatch on pipeline lag; IaC.
- **Security:** short-lived pre-signed URLs; per-user authorization; S3 bucket policies (no public write); encryption at rest/in transit; content moderation.
- **Reliability:** S3 durability; multi-AZ services; async media pipeline retries on failure; feed degrades to pull mode.
- **Performance Efficiency:** CloudFront delivers media from the edge; pre-generated variants; Redis feed cache; direct-to-S3 upload removes an app-tier bottleneck.
- **Cost Optimization:** S3 Intelligent-Tiering for cold media; CDN offload reduces origin cost; serverless transcoding scales to demand.

- **Sustainability:** Tiering/lifecycle deletes/archives cold media; right-sized variants avoid shipping oversized images; edge caching cuts repeated transfer.
-

5. WhatsApp / Chat System



Architecture overview

Problem: Design a real-time 1:1 (and group) messaging system with delivery/read receipts and presence.

1. Functional Requirements

- **Send/receive messages** in real time (1:1 and small groups).
- **Delivery & read receipts** (sent / delivered / read).
- **Presence** (online/last-seen).
- **Message history** (persist and sync across devices).
- *Out of scope:* calls, encryption details (mention E2E), media specifics.

2. Non-Functional Requirements

- **Real-time / low latency:** messages delivered in < 500 ms.
- **Scale:** ~2B users; tens of billions of messages/day; hundreds of millions of concurrent connections.
- **Reliability:** messages must not be lost; **at-least-once delivery** with dedup → effectively exactly-once for the user.
- **Ordering:** messages within a conversation appear in order.
- **High availability.**

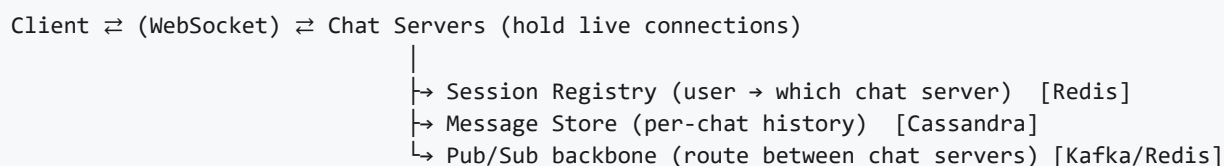
3. Core Entities

- **User, Device, Message** (`messageId`, `chatId`, `senderId`, `content`, `createdAt`, `status`), **Chat/Conversation, Connection** (which server holds a user's live socket).

4. The API (mostly over a persistent connection)

```
WebSocket: connect, send(message), receive(message), ack(delivered/read), typing, presence
REST:      GET /chats/{id}/messages?cursor= (history/sync)
```

5. High-Level Design



- **Persistent connections:** clients hold a **WebSocket** to a chat server; the server can push messages instantly.
- **Routing between servers:** user A (on server 1) messages user B (on server 2). Server 1 looks up B's server in the **session registry** and forwards via the **pub/sub backbone**; B's server pushes it down B's socket.
- **Offline delivery:** if B is offline, persist the message; deliver (and mark delivered) when B reconnects.

6. Potential Deep Dives

- **Connection management at scale:** millions of concurrent sockets → many chat servers, each holding tens/hundreds of thousands of connections. A **session registry** (Redis) maps `userId` → `serverId`. Use a load balancer that supports long-lived connections.
- **Delivery guarantees:** persist the message *before* acking the sender; use **message IDs + client dedup** for at-least-once → exactly-once UX; track status transitions (sent→delivered→read) with acks.
- **Ordering:** assign per-chat sequence numbers (or timestamps + tiebreaker); store and read in order. Kafka partition per chat preserves order.
- **Groups:** fan out to each member's connection/server; for large groups, this is a mini fan-out problem.
- **Presence:** heartbeat over the socket updates a presence store (Redis with TTL); "last seen" = last heartbeat. Presence is best-effort (eventual).
- **Message store:** wide-column store (Cassandra) partitioned by `chatId`, clustered by time — perfect for "recent messages in a chat."

7. Trade-offs

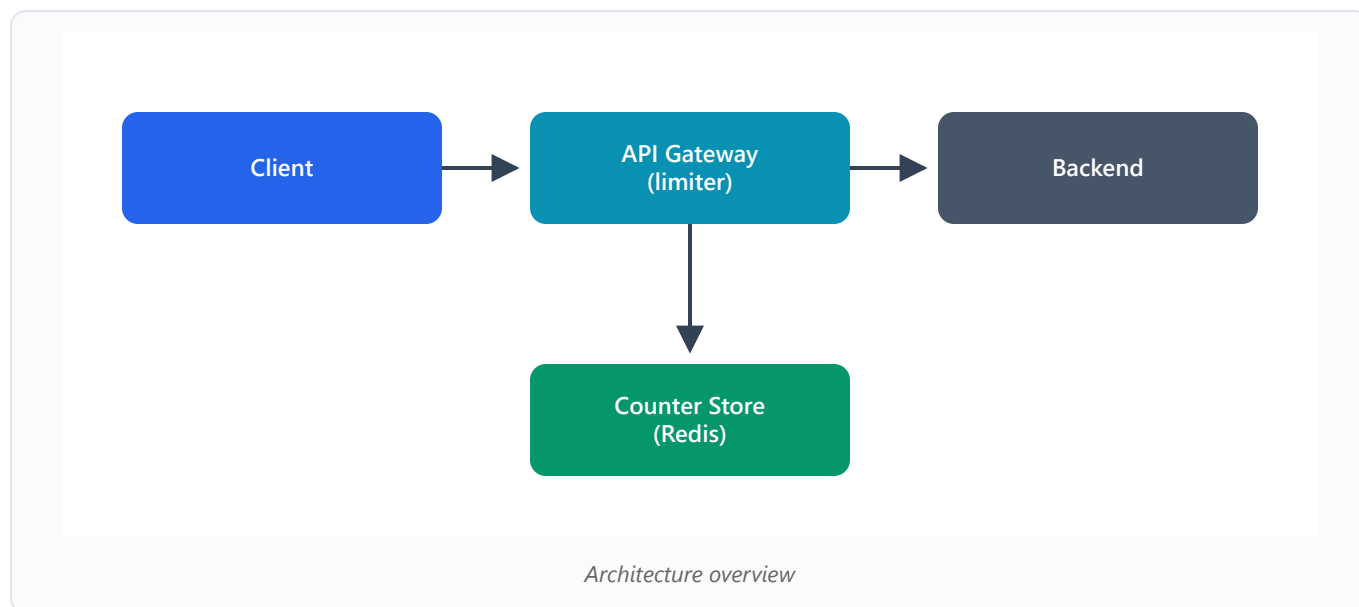
- **WebSocket (stateful) vs polling:** WebSockets give true real-time and efficiency but make servers stateful (harder to scale/balance, need a session registry). Worth it for chat.
- **At-least-once + dedup vs exactly-once:** true exactly-once delivery is very hard; at-least-once + idempotent client dedup is the pragmatic standard.
- **Presence accuracy vs cost:** precise presence is chatty/expensive; we accept eventual, best-effort presence.
- **Store-and-forward:** persisting every message guarantees delivery but adds write load — necessary for reliability.

AWS Well-Architected Assessment

- **Operational Excellence:** managed WebSocket (API Gateway WebSocket) or ECS-managed socket fleet; MSK for the backbone; dashboards on connection count, delivery latency, and queue lag.
 - **Security:** end-to-end encryption for message content; TLS for transport; authN on connect; abuse/spam controls; least-privilege access to stores.
 - **Reliability:** persist-before-ack guarantees no loss; multi-AZ Cassandra/Redis/Kafka; reconnect logic + offline queue; no SPOF in routing.
 - **Performance Efficiency:** pub/sub backbone routes efficiently between servers; Redis session registry for O(1) lookups; partitioned message store for fast history reads.
 - **Cost Optimization:** connection servers right-sized to concurrent sockets; batching/compression of messages; tiered storage for old history.
 - **Sustainability:** efficient long-lived connections vs constant polling saves huge network/compute; cold message history archived; autoscaling connection fleet to actual load.
-

Part B — Questions 6–10

6. Rate Limiter



Problem: Design a service that limits how many requests a client can make in a time window, protecting backends from abuse and overload.

1. Functional Requirements

- **Limit requests** per client (by API key / user / IP) to N per window.
- **Allow or reject** each request (reject with HTTP 429 + `Retry-After`).
- **Configurable rules** per endpoint/tier.
- *Out of scope:* billing, analytics.

2. Non-Functional Requirements

- **Very low latency:** it sits in the request path — must add < 1–2 ms.
- **High throughput:** must handle the full traffic of the system it protects (e.g., 1M RPS).
- **Distributed correctness:** the limit must hold across many app servers (a single global count per client).
- **Availability:** if the limiter fails, decide fail-open (allow) vs fail-closed (deny).

3. Core Entities

- **Rule:** `clientId`, `limit`, `windowSize`, `scope` (endpoint).
- **Counter/State:** per-client usage state (count + timestamps or tokens).

4. The API

```
allow(clientId, endpoint) → { allowed: bool, remaining, retryAfter? }
```

Usually invoked as **middleware** at the API gateway before the request reaches backends.

5. High-Level Design

```
Client → API Gateway (rate-limit middleware) → Backend
          |
          └→ Central store (Redis) holding per-client counters
```

- Each gateway instance checks/updates the client's counter in a **shared Redis** so the limit is global, not per-instance.
- **Algorithm choice** determines the counter shape (see deep dive). Default: **token bucket** (allows bursts, smooth refill).

6. Potential Deep Dives

- **Algorithms:**
- *Token bucket*: tokens refill at a rate; each request consumes one; allows bursts. **Most common.**
- *Leaky bucket*: constant output rate; smooths bursts.
- *Fixed window counter*: simple but has boundary spikes (2× at window edges).
- *Sliding window log/counter*: accurate, avoids boundary problem; counter variant is efficient.
- **Distributed counting**: store counters in **Redis**; use atomic operations (`INCR` , Lua scripts) to avoid race conditions across gateway instances. TTL the keys to the window.
- **Latency**: co-locate Redis with gateways; consider a **local token bucket** with periodic sync to Redis (approximate but ultra-fast) for extreme scale.
- **Fail-open vs fail-closed**: if Redis is unavailable, **fail-open** (allow) is usual for user-facing APIs (availability > perfect enforcement); fail-closed for security-critical limits.
- **Race conditions**: use atomic Redis Lua scripts so check-and-decrement is a single operation.

7. Trade-offs

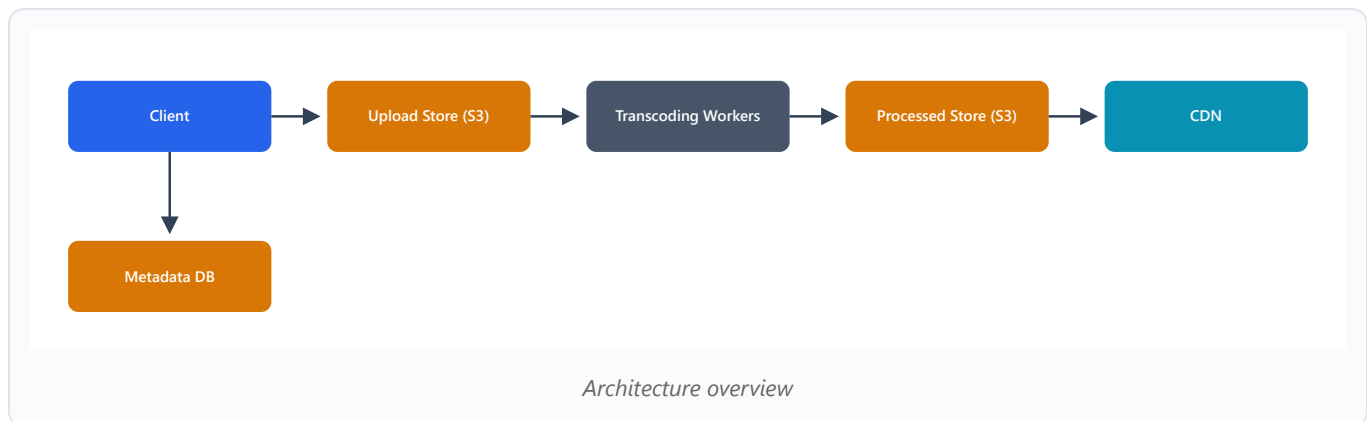
- **Accuracy vs performance**: exact distributed counting (every request hits Redis) is accurate but adds latency/load; local approximate counting is fast but can slightly overshoot. Choose per requirement.
- **Token vs fixed window**: token bucket allows bursts (usually desirable); fixed window is simplest but spiky.
- **Fail-open vs fail-closed**: availability vs strict enforcement — a deliberate business choice.

AWS Well-Architected Assessment

- **Operational Excellence**: use API Gateway usage plans / AWS WAF rate rules where possible (managed); CloudWatch on 429 rates and limiter latency.

- **Security:** rate limiting *is* a security control (anti-DDoS, brute-force); combine with WAF; protect the config store with least-privilege.
 - **Reliability:** ElastiCache (Redis) multi-AZ; defined fail-open/closed behavior; no SPOF in the limiter path.
 - **Performance Efficiency:** in-memory Redis with atomic ops; optional local buckets to shave latency at 1M RPS.
 - **Cost Optimization:** rejecting abusive traffic early saves backend/compute cost; managed WAF/limits avoid custom infra.
 - **Sustainability:** dropping excess load prevents wasteful downstream compute; efficient in-memory counters.
-

7. YouTube / Netflix (Video Streaming)



Problem: Design a system to upload, process, store, and stream video to millions of viewers.

1. Functional Requirements

- **Upload** videos.
- **Process/transcode** into multiple resolutions/formats.
- **Stream/playback** with adaptive quality.
- **Browse/search** (mention, defer).
- *Out of scope:* recommendations, comments.

2. Non-Functional Requirements

- **Extremely read-heavy** (views $\gg$ uploads); massive bandwidth.
- **Scale:** billions of views/day; petabytes of storage.
- **Low startup latency & smooth playback** (no buffering).
- **High availability & durability.**
- *Estimation:* video is the dominant cost — a single 1080p hour $\approx$ several GB $\times$ multiple renditions; global delivery mandates a CDN.

3. Core Entities

- **Video** (`videoId`, `uploaderId`, `title`, `status`, `renditions[]`), **User**, **Rendition** (resolution $\rightarrow$ storage key + manifest), **ViewEvent** (analytics).

4. The API

```
POST /videos/upload-url    → pre-signed upload URL
POST /videos               → { uploadKey, title } → { videoId } (kicks off processing)
GET  /videos/{id}/manifest → adaptive streaming manifest (HLS/DASH)
GET  /videos/{id}          → metadata
```

5. High-Level Design

Upload: Client → pre-signed URL → S3 (raw)
S3 event → Transcoding Pipeline (workers) → renditions → S3 (processed)
Metadata: Video Service → Metadata DB
Playback: Client → Manifest → CDN → video chunks (from S3 origin, cached at edge)

- **Direct-to-S3 upload** (pre-signed), same as Instagram.
- **Transcoding pipeline:** split video into segments, transcode to multiple bitrates/resolutions in parallel (queue + worker fleet), package as **HLS/DASH** (chunked with a manifest).
- **Adaptive bitrate streaming:** the client fetches a manifest and picks the best rendition for its bandwidth, switching dynamically; chunks are served from the **CDN**.

6. Potential Deep Dives

- **Transcoding at scale:** segment-level parallelism (split → transcode chunks concurrently → merge/package); a DAG/workflow orchestrator manages steps and retries; autoscale workers with the upload queue.
- **Adaptive streaming (HLS/DASH):** video is stored as small chunks per bitrate; the manifest lists them; the player switches bitrate per network conditions → smooth playback.
- **CDN is central:** chunks are immutable and highly cacheable; edge caching handles the enormous read bandwidth and keeps startup latency low. Pre-position popular content at edges.
- **Storage tiering:** hot videos on fast tiers/CDN; cold/long-tail in cheaper storage.
- **Resumable uploads:** chunked, resumable uploads for large files/flaky networks.

7. Trade-offs

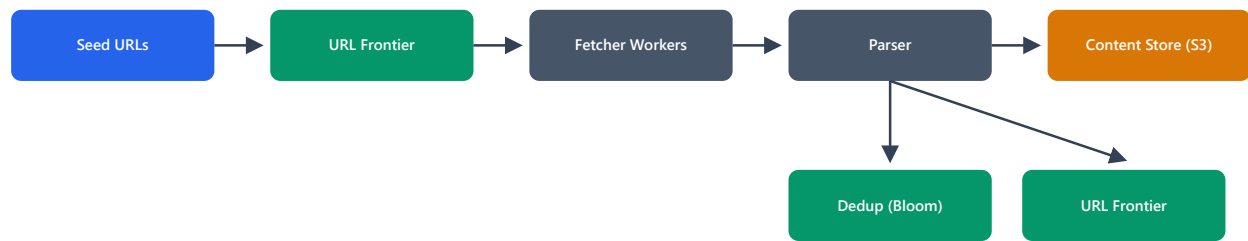
- **Pre-generate all renditions (storage) vs on-the-fly:** pre-generating costs storage but gives instant, cacheable playback; on-the-fly saves storage but adds latency. For read-heavy streaming, **pre-generate**.
- **HLS vs DASH:** device compatibility vs openness; often support both.
- **CDN cost vs performance:** CDN is expensive but non-negotiable for global smooth playback.

AWS Well-Architected Assessment

- **Operational Excellence:** MediaConvert / Step Functions orchestrate transcoding; CloudWatch on pipeline throughput and rebuffer rates; IaC.
- **Security:** signed CDN URLs / DRM for premium content; encryption at rest/in transit; scoped upload URLs; least-privilege pipeline roles.
- **Reliability:** S3 durability; pipeline retries; multi-region CDN (CloudFront) with origin failover; no SPOF.
- **Performance Efficiency:** CloudFront edge delivery + adaptive bitrate; parallel transcoding; right-sized worker instances (GPU/compute-optimized).

- **Cost Optimization:** Spot instances for transcoding; S3 tiering for cold video; CDN offload cuts origin egress; transcode only needed renditions.
 - **Sustainability:** efficient codecs (AV1/H.265) reduce bytes shipped; Spot + autoscaling use spare capacity; edge caching avoids repeated origin transfer.
-

8. Web Crawler



Architecture overview

Problem: Design a system that crawls the web — fetching pages, extracting links, and storing content for indexing.

1. Functional Requirements

- **Crawl** pages starting from seed URLs, following links.
- **Store** fetched content for downstream processing (indexing).
- **Respect politeness** (robots.txt, rate per domain).
- **Avoid duplicates** (don't re-crawl the same content endlessly).
- *Out of scope*: the search index/ranking itself.

2. Non-Functional Requirements

- **Massive scale**: billions of pages; must be highly parallel.
- **Politeness**: don't overload any single domain.
- **Freshness**: re-crawl important pages periodically.
- **Fault tolerance**: resume after failures; no lost work.
- **Extensibility**: handle different content types.

3. Core Entities

- **URL** (url, status, lastCrawled, priority, domain), **Page/Content** (raw HTML blob), **Domain** (politeness state), **URL Frontier** (the queue of URLs to crawl).

4. The API (mostly internal)

```
addSeed(urls)
frontier.next() → nextUrl (respecting politeness)
store(url, content)
```

5. High-Level Design



- **The URL Frontier** is the heart: a set of queues that decide *what to crawl next*, balancing **priority** (importance/freshness) and **politeness** (per-domain rate limits).
- **Fetch** → **parse** → **extract links** → **dedup** → **enqueue** is the core loop, run by a large worker fleet.

6. Potential Deep Dives

- **URL Frontier design:** two-level queues — front queues for **priority**, back queues for **politeness** (one per domain, with a delay), so each domain is crawled at a polite rate while high-priority URLs go first.
- **Deduplication:** track seen URLs (a huge set → Bloom filter + backing store) and **content hashes** (detect identical pages at different URLs) to avoid redundant work/storage.
- **Politeness:** obey `robots.txt` (cache per domain); enforce a min delay between hits to the same host; identify with a user-agent.
- **Distributed coordination:** partition the frontier by domain across workers (a domain is handled by one worker to enforce politeness); use a durable queue so crashes don't lose URLs.
- **Traps & limits:** detect crawler traps (infinite calendars, dynamic URLs), cap depth/size, handle redirects and errors.
- **Freshness:** re-crawl by priority/change-rate (news often, static rarely).

7. Trade-offs

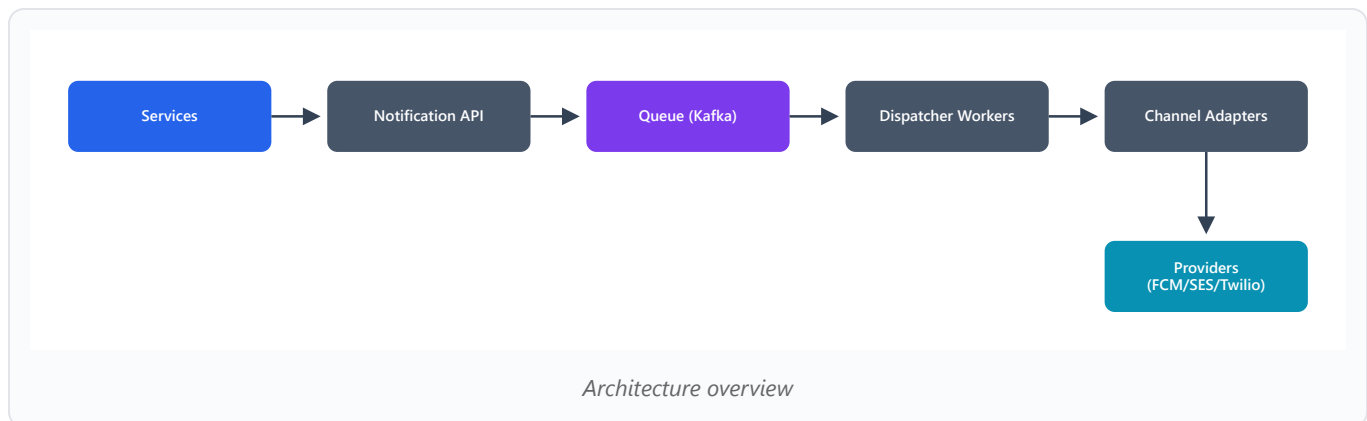
- **Politeness vs speed:** per-domain rate limits slow crawling but are mandatory to be a good citizen; parallelize *across* domains instead.
- **BFS vs priority:** pure BFS is simple; priority crawling fetches important pages first but needs scoring.
- **Bloom filter dedup:** memory-efficient but has false positives (may skip a few new URLs) — acceptable trade for scale.

AWS Well-Architected Assessment

- **Operational Excellence:** SQS-based frontier; autoscaling Fargate/EC2 fetchers; CloudWatch on crawl rate, error rate, queue depth; IaC.

- **Security:** sandbox parsing (untrusted content); egress controls; respect robots.txt (ethical/legal); least-privilege to S3.
 - **Reliability:** durable queues (SQS) so no URL is lost on crash; idempotent fetch/store; retries with backoff; checkpointed frontier.
 - **Performance Efficiency:** massive horizontal parallelism across domains; Bloom-filter dedup; content stored in S3.
 - **Cost Optimization:** Spot instances for fetchers; dedup avoids re-fetching/re-storing; tiered storage for raw HTML.
 - **Sustainability:** dedup + politeness reduce wasted fetches/bandwidth; Spot uses spare capacity; crawl-frequency tuned to change rate avoids needless re-crawls.
-

9. Notification System



Problem: Design a system that sends notifications to users across multiple channels (push, SMS, email).

1. Functional Requirements

- **Send notifications** via push, SMS, and email.
- **Multiple triggers:** other services request a notification for a user.
- **User preferences:** channel opt-in/opt-out, quiet hours.
- **Reliability:** don't lose notifications; avoid duplicates.
- *Out of scope:* rich templating UI, analytics dashboards.

2. Non-Functional Requirements

- **High throughput:** millions–billions/day, bursty.
- **Reliability:** at-least-once delivery + dedup.
- **Low latency** for time-sensitive alerts.
- **Extensible:** add channels/providers easily.
- **Availability.**

3. Core Entities

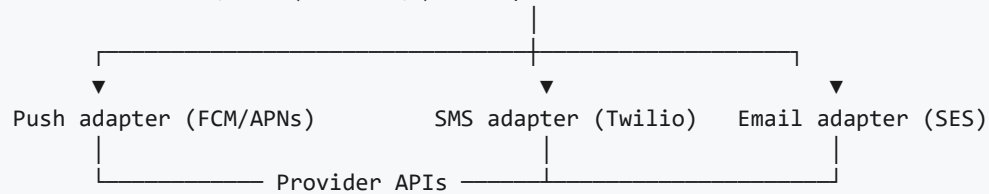
- **Notification** (`id`, `userId`, `channel`, `payload`, `status`), **User Preferences**, **Template**, **Provider** (APNs/FCM/Twilio/SES adapters).

4. The API

```
POST /notifications → { userId, type, data, channels? } → { notificationId }
GET /notifications/{id}/status
PUT /users/{id}/preferences
```

5. High-Level Design

Services → Notification API → Queue (Kafka/SQS) → Dispatcher workers



- **Ingest → queue → dispatch:** requests are validated and enqueued (buffering + reliability), then dispatcher workers apply **user preferences**, render **templates**, and route to the right **channel adapter**.
- **Adapters** abstract each provider so new channels/providers plug in cleanly.

6. Potential Deep Dives

- **Reliability & dedup:** durable queue; **idempotency keys** so the same logical notification isn't sent twice (client/service retries + at-least-once queue = need dedup); dead-letter queue for repeated failures.
- **Fan-out & throughput:** partition the queue for parallelism; rate-limit per provider (they have their own limits); batch where providers allow.
- **User preferences & quiet hours:** check opt-outs/quiet hours before sending; schedule delayed sends.
- **Prioritization:** separate queues for high-priority (OTP, security) vs bulk (marketing) so urgent ones aren't stuck behind a marketing blast.
- **Provider failover:** if one SMS provider fails, fail over to a backup; track delivery receipts/callbacks to confirm delivery.
- **Templating:** server-side templates with localization.

7. Trade-offs

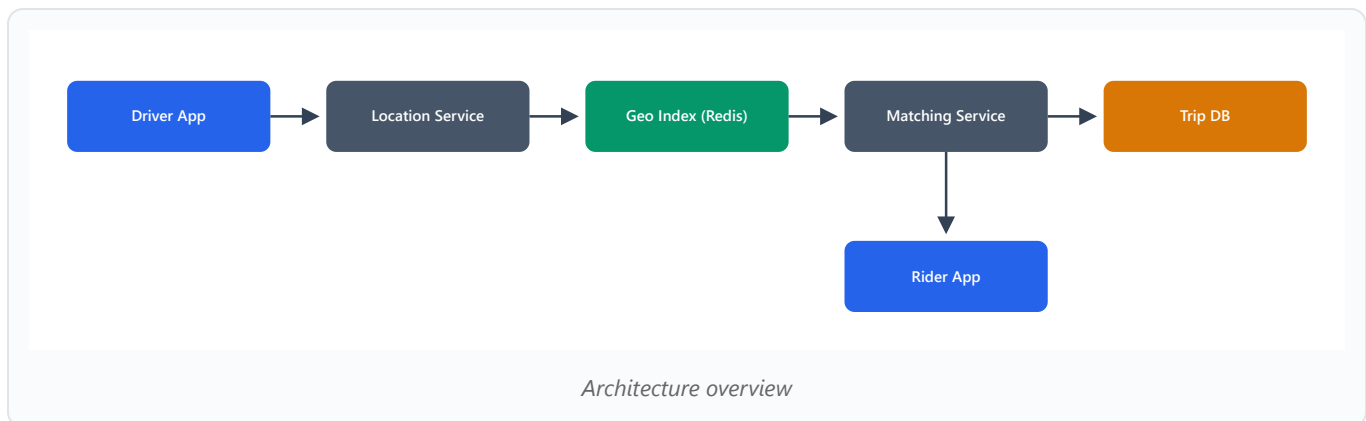
- **At-least-once + dedup vs exactly-once:** exactly-once is impractical across third-party providers; at-least-once + idempotency is the standard.
- **Sync vs async:** async (queue) decouples callers, absorbs bursts, and enables retries — at the cost of immediate delivery confirmation (use status callbacks).
- **Priority queues vs single queue:** separate queues add complexity but protect urgent traffic.

AWS Well-Architected Assessment

- **Operational Excellence:** SNS/SES + SQS provide managed building blocks; CloudWatch on delivery success/latency per channel; DLQs monitored; IaC.
- **Security:** protect PII (phone/email) with encryption; least-privilege to provider credentials (Secrets Manager); consent/opt-out enforcement (compliance).
- **Reliability:** durable queues + retries + DLQ; provider failover; idempotent delivery; multi-AZ.
- **Performance Efficiency:** partitioned queues for parallel dispatch; batching; priority lanes for urgent messages.

- **Cost Optimization:** batching and preference filtering avoid needless (paid) SMS/email sends; SQS/SNS scale to demand; suppress bounces.
 - **Sustainability:** filtering unwanted/opted-out sends reduces wasted work and provider load; autoscaling dispatchers to actual volume.
-

10. Uber / Ride-Sharing



Problem: Design the core of a ride-hailing app — match riders to nearby drivers and track the trip.

1. Functional Requirements

- **Riders request a ride** (pickup + destination).
- **Match** the rider to a nearby available driver.
- **Real-time location** tracking during the trip.
- **Trip lifecycle:** request → matched → in-progress → completed.
- *Out of scope:* payments (see Q18), ratings, surge pricing details.

2. Non-Functional Requirements

- **Low latency matching** (seconds) and real-time location updates.
- **Scale:** millions of drivers/riders; high-frequency location pings (every few seconds).
- **High availability** (a city outage = lost revenue and stranded users).
- **Consistency:** a driver must be matched to **one** rider at a time (no double-booking).
- **Geospatial** queries at scale.

3. Core Entities

- **Rider, Driver** (`driverId`, `location`, `status`), **Trip** (`tripId`, `riderId`, `driverId`, `status`, `route`), **Location** (frequent updates).

4. The API

```
POST /rides          → { pickup, destination } → { tripId, status }
POST /drivers/location → { driverId, lat, lng } (frequent updates)
WebSocket            → live trip/location updates to rider & driver
```

5. High-Level Design

Driver app → Location Service → Geospatial index (Redis Geo / quadtree) [driver locations]
Rider app → Ride/Matching Service → query nearby drivers → offer → Trip Service → Trip DB
Both ⇌ WebSocket gateway ⇌ real-time updates

- **Location ingestion:** drivers stream location; stored in a **geospatial index** optimized for "find drivers near (lat,lng)".
- **Matching:** on a ride request, query nearby available drivers, rank (ETA/distance), offer to the best; on accept, create the **Trip** and lock that driver.
- **Real-time updates** flow over WebSockets to both parties.

6. Potential Deep Dives

- **Geospatial indexing:** use **geohash / quadtree / S2 cells** (or Redis GEO) to index driver locations so "nearby" queries are fast. Geohash buckets nearby points into shared prefixes → efficient range lookups. Rebalance cell size by density (dense cities vs rural).
- **High-frequency location writes:** millions of drivers pinging every few seconds = huge write volume → keep current location in an **in-memory store** (Redis), not a disk DB; persist trip routes separately/asynchronously.
- **Matching consistency:** when offering a driver, **atomically** mark them "reserved" so two riders can't grab the same driver (Redis lock / conditional update). Handle decline/timeout → offer next driver.
- **Scaling by geography:** partition by city/region (natural sharding) — location data and matching are inherently local, so shard by geo cell/region.
- **Real-time connections:** WebSocket gateway with a session registry (like the chat system) to push updates.

7. Trade-offs

- **In-memory vs durable location store:** in-memory (Redis) gives the needed write throughput/low latency; the trade is durability (fine — current location is ephemeral; trips are persisted).
- **Geohash precision:** finer cells = more precise but more buckets to query at borders; coarser = fewer queries but less precise. Tune per density.
- **Matching optimality vs speed:** the globally optimal match is expensive; a fast "good enough nearest" match wins for UX.
- **Consistency for matching (strong) vs everything else (eventual):** strong only where it matters (driver reservation).

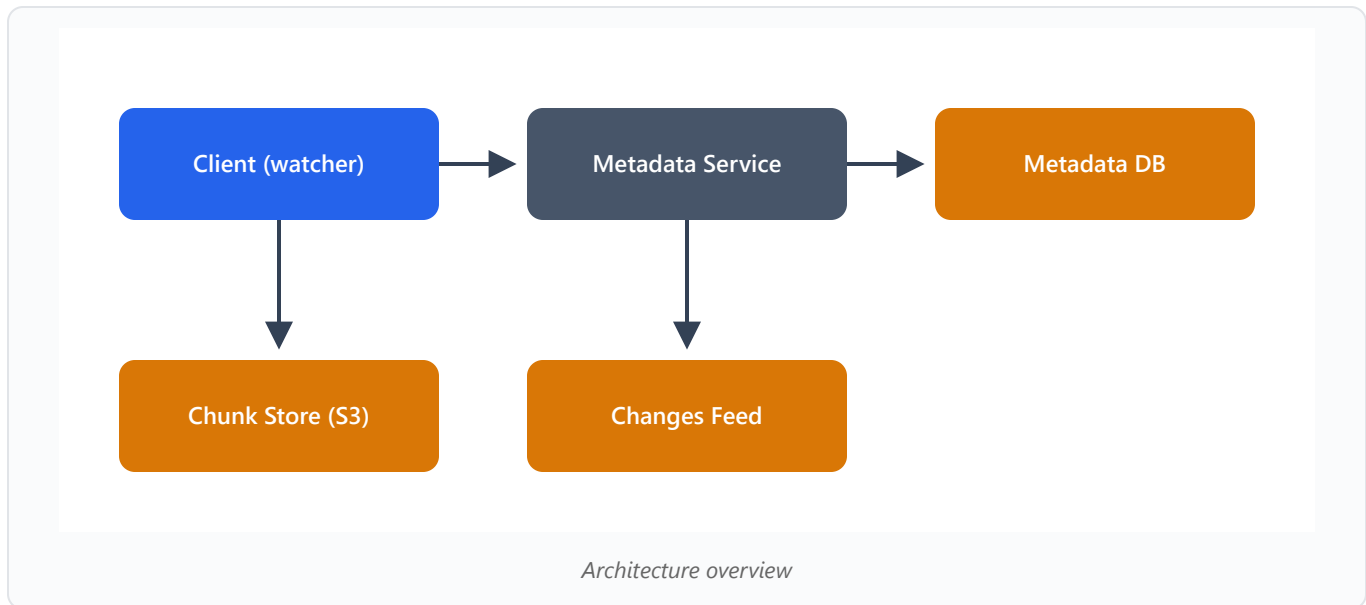
AWS Well-Architected Assessment

- **Operational Excellence:** ElastiCache (Redis Geo), managed WebSocket/API Gateway; CloudWatch on match latency and location ingest rate; regional deployments; IaC.
- **Security:** authN on all apps; protect precise location (PII) with encryption + strict access; least-privilege; audit trip data access.
- **Reliability:** multi-AZ Redis with replicas; regional isolation limits blast radius; trip state persisted durably; reconnect handling for sockets.

- **Performance Efficiency:** in-memory geo index for fast nearby queries; geo-sharding keeps queries local; WebSockets for real-time push.
 - **Cost Optimization:** in-memory only for hot/current location (small, ephemeral); async persistence; autoscale by regional demand (rush-hour peaks).
 - **Sustainability:** geo-partitioning avoids global scans; efficient location updates (deltas/throttled pings) cut network/compute; scale to demand.
-

Part C — Questions 11–15

11. Google Drive / Dropbox (File Storage & Sync)



Problem: Design a cloud file storage service that stores files and syncs them across a user's devices.

1. Functional Requirements

- **Upload/download** files.
- **Sync** changes across all of a user's devices.
- **Share** files/folders with other users.
- **Versioning** (keep file history).
- *Out of scope:* real-time collaborative editing.

2. Non-Functional Requirements

- **Durability first:** never lose a file (11 nines).
- **Scale:** hundreds of millions of users; exabytes of data; large files.
- **Efficient sync:** upload only what changed, not whole files.
- **Availability & consistency:** a user should see a consistent view of their files.
- *Estimation:* files are large and numerous → object storage; sync efficiency drives the design.

3. Core Entities

- **User**, **File** (`fileId`, `name`, `ownerId`, `size`, `version`), **Chunk** (a piece of a file + hash), **Device**, **Share/Permission**, **FileMetadata/Version**.

4. The API

```
POST /files/{id}/chunks    → upload changed chunks (by hash)
GET  /files/{id}           → metadata + chunk list
GET  /chunks/{hash}        → download a chunk
POST /files/{id}/share     → grant access
GET  /changes?since=cursor → sync: what changed for this user
```

5. High-Level Design

```
Client (watcher) → chunk & hash file → upload changed chunks → S3 (chunk store)
                  → update Metadata Service → Metadata DB (files, versions, chunk lists)
Sync: Client polls/subscribes to /changes → pulls new chunks → reassembles file
```

- **Chunking:** files are split into fixed/variable-size **chunks**, each identified by a **content hash**. Only chunks whose hash changed are uploaded (**delta sync**) → massive bandwidth savings.
- **Metadata vs content split:** chunks live in **object storage**; the **metadata DB** tracks files → ordered chunk hashes, versions, and permissions.
- **Sync:** a client watches for local changes and polls a **changes feed** for remote changes, downloading only new chunks.

6. Potential Deep Dives

- **Delta sync via chunking + dedup:** content-addressed chunks mean identical chunks (across versions or even users) are stored once (**dedup**) and never re-uploaded. Editing 1 MB of a 1 GB file uploads ~1 MB.
- **Conflict resolution:** two devices edit offline → conflict. Strategies: last-write-wins, or keep both as conflicted copies (Dropbox's approach) — never silently lose data. Use version vectors to detect conflicts.
- **Metadata consistency:** the metadata DB is the source of truth for "what the file looks like now"; make metadata updates transactional so a file version is atomic (all its chunks referenced together).
- **Sync efficiency:** a **changes/journal feed** per user (cursor-based) lets clients pull only deltas; use notifications (long-poll/WebSocket) for near-real-time sync.
- **Large file uploads:** resumable, parallel chunk uploads.

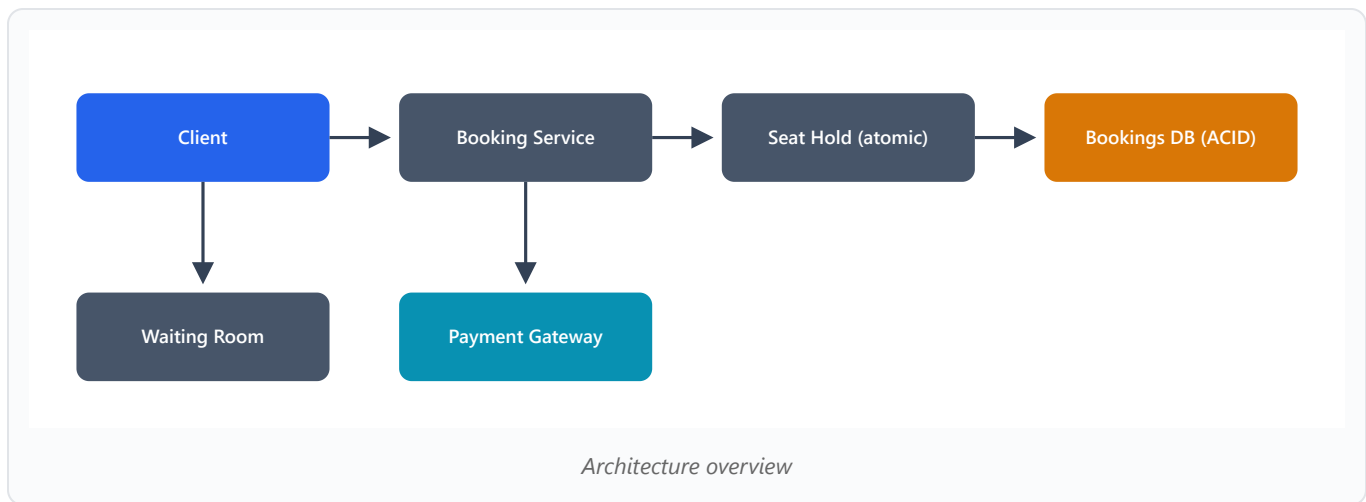
7. Trade-offs

- **Fixed vs variable-size chunking:** fixed is simple but a byte insertion shifts all chunks (poor dedup); **content-defined (variable) chunking** dedups better across edits but is more complex.
- **LWW vs conflict copies:** LWW is simple but can lose edits; conflict copies preserve data but add user friction. Preserve data.
- **Metadata DB choice:** strong consistency for metadata (SQL/transactional) vs scale — often a transactional metadata store + object storage for scale.

AWS Well-Architected Assessment

- **Operational Excellence:** S3 for chunks + managed metadata DB; CloudWatch on sync lag and upload success; IaC; automated version cleanup.
 - **Security:** per-file encryption at rest; scoped access (pre-signed URLs); robust sharing/permission model; audit access; TLS.
 - **Reliability:** S3 11-nines durability + versioning; multi-AZ metadata DB; atomic metadata updates; resumable uploads survive network failures.
 - **Performance Efficiency:** delta sync + dedup minimize transfer; CDN for downloads; parallel chunk I/O.
 - **Cost Optimization:** dedup stores each chunk once; S3 tiering for cold files/old versions; only-changed-chunk uploads cut bandwidth cost.
 - **Sustainability:** dedup + delta sync drastically reduce stored bytes and network transfer; lifecycle-archive old versions.
-

12. Ticketmaster (Ticket Booking)



Problem: Design a system to browse events and book seats, preventing double-booking under high concurrency (e.g., a hot concert on-sale).

1. Functional Requirements

- **Browse events** and view seat availability.
- **Reserve/hold** a seat during checkout.
- **Purchase** the held seat.
- **Release** holds that expire without purchase.
- *Out of scope:* recommendations, dynamic pricing details.

2. Non-Functional Requirements

- **Strong consistency for booking:** a seat is sold to **exactly one** buyer (no double-booking) — this is the crux.
- **High concurrency / spiky:** thousands compete for the same seats at on-sale time.
- **Read-heavy browsing** but critical-write booking.
- **Availability** for browsing; correctness for buying.

3. Core Entities

- **Event** (eventId , venue , datetime), **Seat** (seatId , eventId , status : available/held/booked), **Reservation/Hold** (seatId , userId , expiresAt), **Booking/Order**, **User**.

4. The API

```
GET /events/{id}/seats          → seat map + availability
POST /events/{id}/holds         → { seatIds } → hold seats (temporary)
```

POST /bookings	→ confirm purchase of held seats
DELETE /holds/{id}	→ release

5. High-Level Design

Browse: Client → LB → Event Service → Cache (seat availability) → Event DB
Book: Client → Booking Service → (atomic seat hold) → Reservation store → Payment → confirm
|
Hold expiry job (releases unpaid holds)

- **Hold-then-purchase:** to avoid taking payment for a seat someone else grabs, first **atomically hold** the seat (status available→held with a TTL), then process payment, then finalize (held→booked). If payment fails or the hold expires, release it.
- **The critical section** is the atomic seat state transition — must prevent two users holding the same seat.

6. Potential Deep Dives

- **Preventing double-booking (the core):** the seat hold must be **atomic and strongly consistent**. Options:
 - *DB transaction with row lock / conditional update:* `UPDATE seats SET status='held' WHERE seatId=? AND status='available'` — succeeds for only one user.
 - *Distributed lock (Redis) per seat* for the hold window.
 - Use an ACID database for the seat/booking state — correctness over scale here.
- **Hold expiry:** store `expiresAt`; a background job (or TTL) releases expired holds back to available. Handle the race between expiry and late payment.
- **Handling on-sale spikes:** a **virtual waiting room / queue** admits users gradually so the booking DB isn't overwhelmed; browse traffic served from cache/CDN; only the booking path touches the strongly-consistent store.
- **Read vs write split:** availability views can be slightly stale (cached); the *hold/purchase* path is strongly consistent.
- **Idempotency:** booking requests carry idempotency keys (retries mustn't double-book or double-charge).

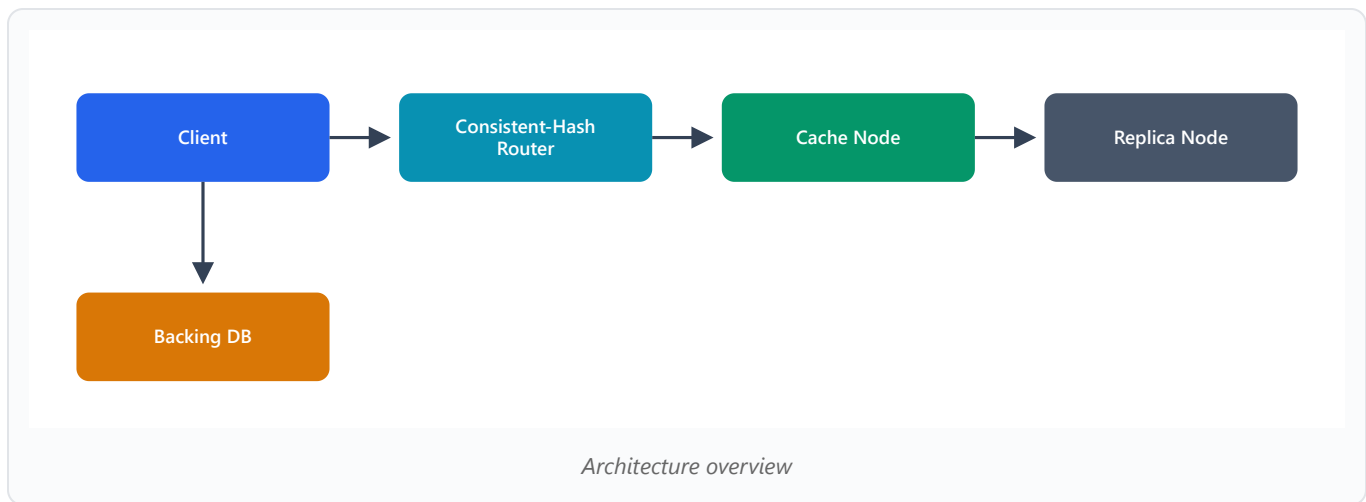
7. Trade-offs

- **Strong consistency vs scale:** the booking path sacrifices some scalability for correctness (locks/transactions) — the right call; scale the *browse* path separately.
- **Optimistic vs pessimistic locking:** optimistic (conditional update) scales better under low contention; pessimistic (locks) is safer under the extreme contention of a hot on-sale. Often optimistic with retries.
- **Hold TTL length:** longer holds = better UX but seats locked away from others; shorter = more churn. Tune (e.g., 5–10 min).
- **Waiting room:** adds friction but protects the system and ensures fairness.

AWS Well-Architected Assessment

- **Operational Excellence:** managed ACID DB (Aurora); queue-based waiting room; CloudWatch on hold conflicts, booking latency; game-day tests for on-sales.
 - **Security:** authN; idempotent, auditable transactions; PCI-compliant payment handoff; least-privilege; fraud checks.
 - **Reliability:** multi-AZ Aurora; the strongly-consistent core prevents double-booking; holds auto-release on failure; idempotency prevents duplicate orders.
 - **Performance Efficiency:** cache/CDN for browsing; waiting room throttles write load to the DB's capacity; conditional updates avoid heavy locking where possible.
 - **Cost Optimization:** scale the read path cheaply (cache) and reserve strong-consistency resources for the booking path; autoscale for on-sale spikes then scale down.
 - **Sustainability:** waiting room smooths spikes (avoid massive over-provisioning); cached browsing reduces DB load; scale to event schedule.
-

13. Distributed Cache



Problem: Design a distributed in-memory cache (like Redis/Memcached at scale) that many services can share.

1. Functional Requirements

- **get(key), set(key, value, ttl), delete(key).**
- **Scale horizontally** across many nodes.
- **Eviction** when memory is full.
- **Optional:** replication for availability.

2. Non-Functional Requirements

- **Very low latency** (sub-millisecond).
- **High throughput** (millions of ops/sec).
- **Scalable capacity** (add nodes to grow).
- **Availability:** tolerate node failures.
- **Consistency:** typically eventual/best-effort (it's a cache).

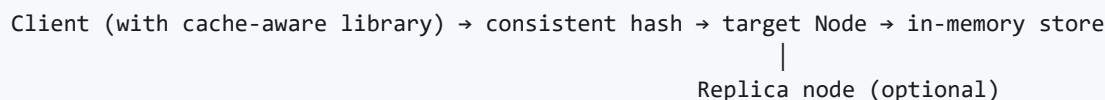
3. Core Entities

- **Entry** (key , value , ttl , lastAccess), **Node** (a cache server holding a partition), **Ring** (consistent-hash mapping of keys→nodes).

4. The API

```
get(key) → value | miss
set(key, value, ttl)
delete(key)
```

5. High-Level Design



- **Partitioning via consistent hashing:** keys are distributed across nodes using **consistent hashing** so adding/removing a node remaps only a small fraction of keys (not all).
- **Client-side routing:** clients (or a proxy) hash the key to find the right node → one network hop → in-memory lookup.
- **Eviction (LRU) + TTL** manage memory per node.

6. Potential Deep Dives

- **Consistent hashing + virtual nodes:** place nodes on a hash ring; each physical node owns many **virtual nodes** for even distribution and smooth rebalancing when nodes join/leave. This is the key to horizontal scaling without mass remapping.
- **Replication & availability:** replicate each partition to N replicas (leader + followers). On node failure, a replica takes over. Choose sync (consistent, slower) vs async (fast, may lose recent writes) replication.
- **Eviction policies:** LRU (default), LFU, or TTL-based; per-node memory bound. Approximate LRU (sampling) is common for performance.
- **Hot keys:** a single very hot key can overload its node → replicate hot keys to multiple nodes, or add a small client-side local cache (L1) in front.
- **Cache coherence:** on writes to the backing DB, invalidate/update cache (write-through or explicit invalidation); accept eventual consistency.
- **Thundering herd:** request coalescing + TTL jitter + stale-while-revalidate on misses.

7. Trade-offs

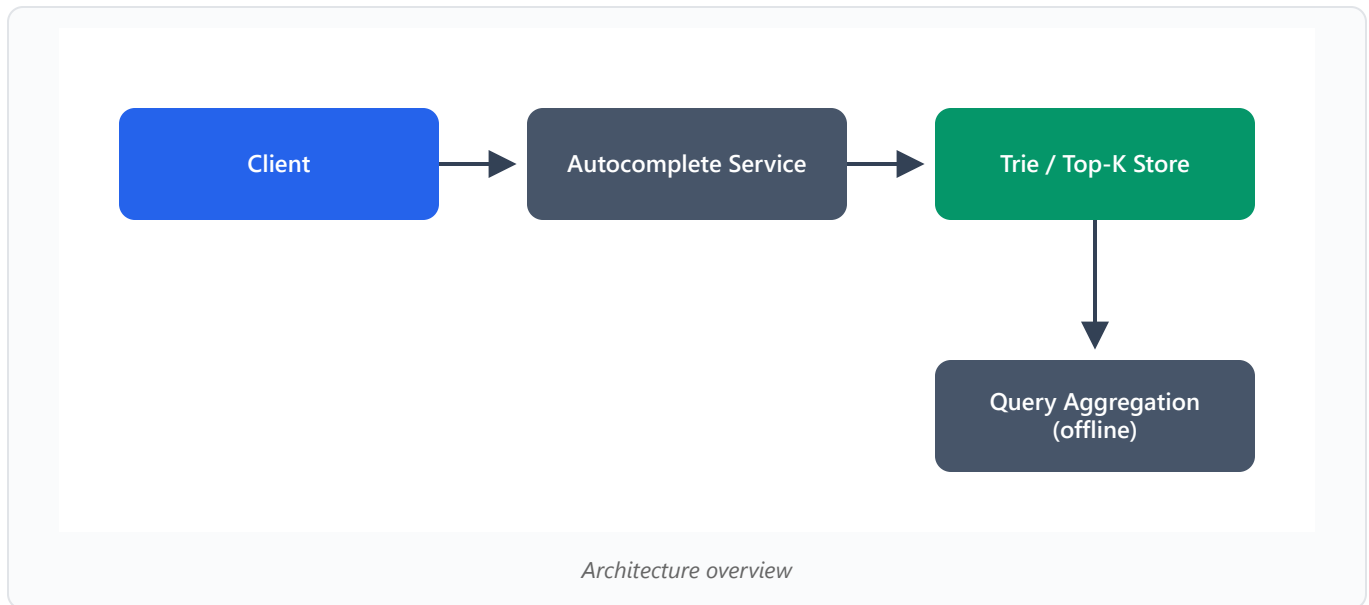
- **Consistency vs performance:** async replication and eventual consistency give speed/availability; a cache rarely needs strong consistency.
- **Client-side routing vs proxy:** client-side is one fewer hop (faster) but couples clients to topology; a proxy centralizes routing at a small latency cost.
- **Replication cost:** replicas improve availability but cost memory; tune replica count to the availability need.
- **Memory vs hit rate:** more memory = higher hit rate = less backend load; balance cost vs benefit.

AWS Well-Architected Assessment

- **Operational Excellence:** ElastiCache (managed Redis/Memcached) handles clustering, failover, patching; CloudWatch on hit rate, evictions, latency; IaC.
- **Security:** in-transit/at-rest encryption; VPC isolation; AUTH; least-privilege; not internet-exposed.

- **Reliability:** multi-AZ replicas + automatic failover; consistent hashing limits rebalancing blast radius; clients degrade to DB on miss/outage.
 - **Performance Efficiency:** in-memory sub-ms access; consistent hashing for even load; L1 local caches for hot keys.
 - **Cost Optimization:** right-size node memory; higher hit rate reduces expensive DB load; reserved nodes for steady baseline.
 - **Sustainability:** caching avoids recomputation/DB work (fewer cycles per request); right-sized memory; eviction keeps only useful data.
-

14. Search Autocomplete / Typeahead



Problem: Design the autocomplete that suggests top queries as a user types, with very low latency.

1. Functional Requirements

- **Suggest** top-K completions for a given prefix.
- **Rank** suggestions (by popularity/frequency).
- **Update** suggestions as new queries trend.
- *Out of scope:* full search itself, personalization (mention).

2. Non-Functional Requirements

- **Ultra-low latency:** suggestions must appear within ~50–100 ms as the user types.
- **Read-heavy:** every keystroke is a query.
- **Scale:** billions of queries; huge prefix space.
- **Freshness:** trending terms should appear (near-real-time-ish, not instant).

3. Core Entities

- **Term/Query** (`text` , `frequency`), **Trie node** (prefix → top suggestions), **PrefixTopK** (precomputed top-K per prefix).

4. The API

```
GET /autocomplete?prefix=har&limit=10 → ["harry potter", "hard drive", ...]
```

5. High-Level Design

Query logs → aggregation (counts) → build Trie / precomputed top-K per prefix
Serving: Client → Autocomplete Service → Trie/top-K store (in-memory) → suggestions

- **Trie (prefix tree):** stores terms so that walking the prefix path yields matching completions. To make it fast, **precompute and cache the top-K suggestions at each node**, so a lookup is: walk to the prefix node → return its cached top-K (no scanning subtrees).
- **Two pipelines:** an **offline pipeline** aggregates query frequencies and (re)builds the trie/top-K; an **online serving layer** answers prefix queries from an in-memory structure.

6. Potential Deep Dives

- **Precomputed top-K per prefix:** naively, finding top-K under a prefix means traversing the whole subtree per keystroke — too slow. Instead, **store the top-K completions directly on each trie node** (updated offline). Lookup becomes $O(\text{prefix length})$. This is the key latency trick.
- **Building/updating the trie:** aggregate query counts from logs (batch, e.g., hourly/daily via MapReduce/Spark); rebuild or incrementally update the trie; ship the new structure to serving nodes. Trending terms appear after the next build (near-real-time via a faster streaming path if needed).
- **Serving at scale:** keep the trie in memory on serving nodes; **shard by prefix** (e.g., by first letters) across nodes; replicate for read throughput; front with a cache/CDN for the hottest prefixes.
- **Ranking:** frequency-based, optionally time-decayed (recent popularity), optionally personalized/localized.
- **Data size:** the trie can be large → shard and compress; only store top-K per node (not all completions).

7. Trade-offs

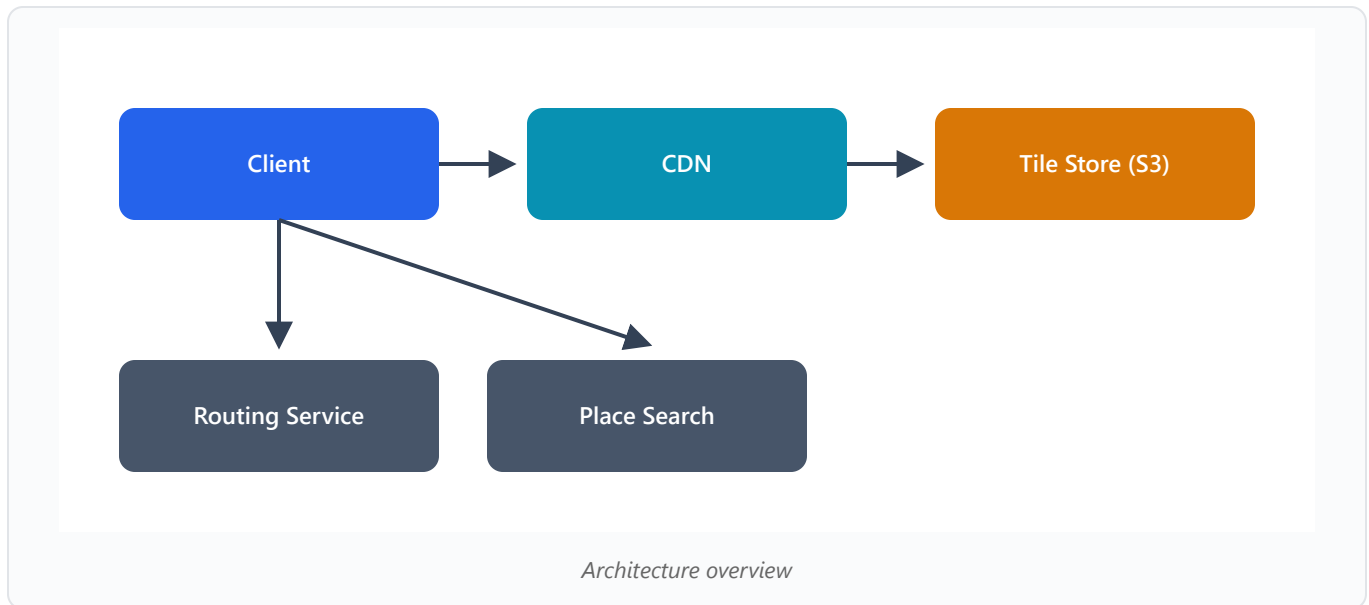
- **Precompute (storage) vs compute-on-read:** precomputing top-K per node uses more memory/build time but gives the required read latency — the right trade for a read-heavy, latency-critical feature.
- **Freshness vs cost:** rebuilding more often surfaces trends faster but costs more compute; batch (periodic) rebuild is usually enough, with an optional streaming layer for hot trends.
- **Trie vs other structures:** tries give natural prefix matching; alternatives (ternary search trees, FSTs) trade memory vs speed.

AWS Well-Architected Assessment

- **Operational Excellence:** offline build on EMR/Glue; versioned trie artifacts shipped to serving nodes; CloudWatch on suggestion latency and build success; IaC.
- **Security:** sanitize/filter suggestions (no offensive/PII leakage from query logs); TLS; least-privilege on logs.

- **Reliability:** immutable, versioned trie artifacts (roll back a bad build); replicated serving nodes; cache fallback; multi-AZ.
 - **Performance Efficiency:** in-memory precomputed top-K $\rightarrow$ O(prefix) lookups; prefix sharding; CDN/cache for hot prefixes.
 - **Cost Optimization:** batch builds on Spot/EMR; serve from memory (no per-keystroke DB hits); cache hottest prefixes to cut serving cost.
 - **Sustainability:** precompute-once/serve-many amortizes compute; batch aggregation on spare capacity; caching avoids repeated work per keystroke.
-

15. Google Maps (Geospatial + Routing)



Problem: Design core mapping features — render the map, find places, and compute driving routes with ETAs.

1. Functional Requirements

- **Render map tiles** for a location/zoom.
- **Search places** (points of interest).
- **Routing:** shortest/fastest path between two points, with ETA.
- **Live traffic** affecting routes/ETA (mention).
- *Out of scope:* street view, indoor maps.

2. Non-Functional Requirements

- **Low latency** for tiles and routing.
- **Massive scale:** billions of requests; planet-scale data.
- **Read-heavy** (tiles/routes read far more than the map changes).
- **Freshness** for traffic/ETA.

3. Core Entities

- **Tile** (`z/x/y` → image/vector), **Place/POI** (`id`, `location`, `metadata`), **RoadGraph** (nodes = intersections, edges = road segments with weights), **TrafficData** (edge → current speed).

4. The API

```
GET /tiles/{z}/{x}/{y}           → map tile
GET /places?q=&near=lat,lng      → place search
GET /route?from=&to=&mode=       → { path, distance, eta }
```

5. High-Level Design

```
Tiles: Client → CDN → Tile store (pre-rendered tiles in object storage)
Places: Client → Place Service → Geospatial index (search by location/name)
Route: Client → Routing Service → Road Graph (in-memory/partitioned) + Traffic → path + ETA
```

- **Tiles are pre-rendered** at multiple zoom levels and served from a **CDN** (immutable, highly cacheable) — this handles the bulk of read traffic cheaply.
- **Routing** runs a shortest-path algorithm over a **road graph** weighted by distance/time, adjusted by **live traffic**.
- **Place search** uses a **geospatial index** for "near me" and name queries.

6. Potential Deep Dives

- **Routing at planet scale:** plain Dijkstra over billions of edges is too slow. Use **graph preprocessing / hierarchical routing** (Contraction Hierarchies, highway hierarchies) — precompute shortcuts so long routes are computed in milliseconds. Partition the graph by region; route within/between partitions.
- **Geospatial indexing: geohash / S2 / quadtree** to index places and map regions for fast spatial queries (Q10 shares this idea). Tiles are naturally partitioned by `z/x/y`.
- **Live traffic & ETA:** ingest GPS/probe data → estimate current speed per road segment → update edge weights → routes/ETAs reflect traffic. This is a streaming aggregation pipeline.
- **Tile serving:** pre-render popular tiles; vector tiles allow client-side rendering (less data); CDN caches aggressively.
- **Scale:** everything is read-heavy and cacheable except routing (compute) and traffic (streaming).

7. Trade-offs

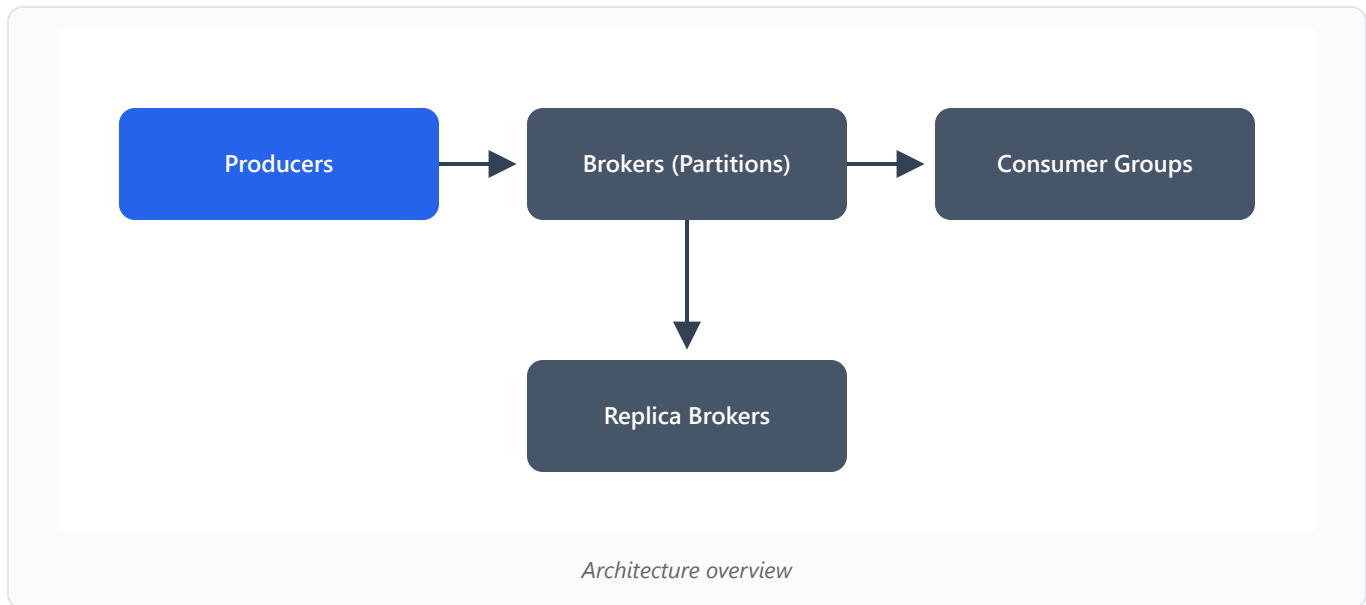
- **Pre-rendered raster vs vector tiles:** raster is simple and cacheable but heavy and zoom-fixed; vector tiles are smaller and flexible (client renders) but need client compute. Often vector for modern apps.
- **Precomputed routing (Contraction Hierarchies) vs on-demand Dijkstra:** preprocessing costs build time/storage but makes queries fast; essential at scale.
- **ETA accuracy vs freshness cost:** more frequent traffic updates = better ETAs but more streaming compute.
- **Consistency:** map/traffic data is eventually consistent — acceptable.

AWS Well-Architected Assessment

- **Operational Excellence:** tile pipelines and graph preprocessing automated; traffic streaming via Kinesis/MSK; CloudWatch on route latency and tile hit rate; IaC.
 - **Security:** API keys + quotas; protect user location/trip data (PII); TLS; least-privilege to data stores.
 - **Reliability:** CDN + object storage for tiles (highly durable/available); partitioned routing service (regional isolation); graceful degradation (stale traffic if the stream lags).
 - **Performance Efficiency:** CDN for tiles; contraction hierarchies for fast routing; geospatial indexes; in-memory hot graph partitions.
 - **Cost Optimization:** CDN offload for the dominant tile traffic; precomputed routing avoids repeated heavy compute; Spot/batch for preprocessing; tiered storage for rarely-viewed tiles.
 - **Sustainability:** massive CDN cache hit rates avoid recompute/transfer; precompute-once routing; vector tiles cut bytes shipped.
-

Part D — Questions 16–20

16. Distributed Message Queue (Kafka-like)



Problem: Design a durable, scalable message queue/streaming platform that decouples producers from consumers.

1. Functional Requirements

- **Produce** messages to a topic.
- **Consume** messages from a topic (one or many independent consumers).
- **Durability:** messages persist and aren't lost.
- **Ordering** (at least within a partition).
- *Out of scope:* stream processing/SQL layer.

2. Non-Functional Requirements

- **High throughput:** millions of messages/sec.
- **Durability:** persisted, replicated; survive broker failure.
- **Scalability:** add brokers/partitions to grow.
- **Delivery semantics:** at-least-once (configurable toward exactly-once).
- **Low latency** for real-time consumers.

3. Core Entities

- **Topic** (a named stream), **Partition** (ordered, append-only log; unit of parallelism), **Message/Record** (offset, key, value, timestamp), **Broker** (server holding partitions), **Consumer Group** (set of consumers sharing a subscription), **Offset** (a consumer's position).

4. The API

```
produce(topic, key, value)      → appended to a partition
subscribe(topic, consumerGroup)
poll() → records
commit(offset)                → mark progress
```

5. High-Level Design

```
Producers → (partition by key) → Brokers [Partition = append-only log on disk]
                                         | replicated to follower brokers
Consumers (in groups) → poll partitions → track offsets
```

- **The log abstraction:** each partition is an **append-only, ordered log** persisted to disk. Producers append; consumers read sequentially by **offset**. Retention keeps messages for a configured time (enabling replay).
- **Partitioning** gives parallelism and ordering-within-partition; **consumer groups** split partitions among consumers (queue-like) while different groups each get all messages (pub/sub).
- **Replication** across brokers gives durability and failover.

6. Potential Deep Dives

- **Durability & replication:** each partition has a **leader** + follower replicas. Producers write to the leader; followers replicate. A write is acknowledged after reaching the leader (and optionally a quorum of replicas — tunable `acks`). On leader failure, a follower is promoted (in-sync replica set).
- **Ordering:** guaranteed **within a partition** only. To keep related messages ordered, produce them with the same **key** (same key → same partition). Global ordering across partitions isn't provided (it would kill scalability).
- **Delivery semantics:** at-least-once (consumer commits offset after processing → crash before commit = reprocessing); exactly-once via idempotent producers + transactional writes. Consumers should be **idempotent**.
- **High throughput tricks:** sequential disk I/O (append-only log is fast), zero-copy transfer, batching, and compression.
- **Scaling:** add partitions for more parallelism (but repartitioning changes key→partition mapping); add brokers to spread partitions. Offsets stored durably (in a special topic).
- **Backpressure/retention:** consumers read at their own pace; slow consumers just lag (bounded by retention), not block producers.

7. Trade-offs

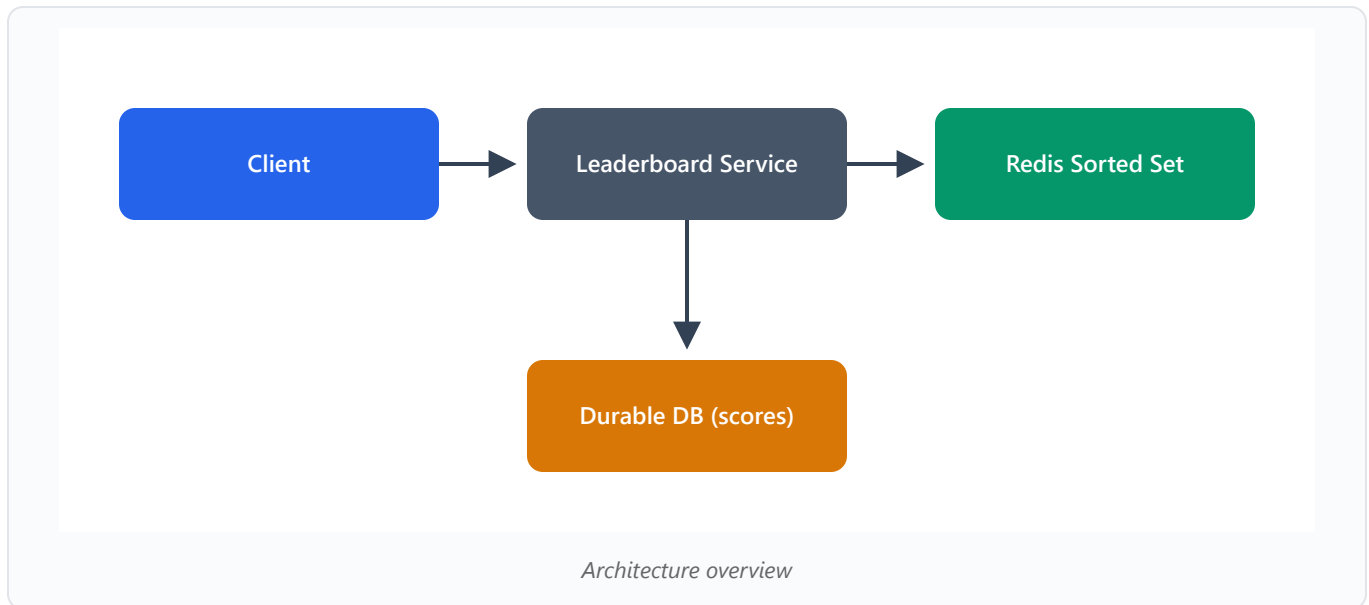
- **Ordering vs scalability:** ordering only within a partition is the price of horizontal scale; global ordering would serialize everything.
- **Latency vs durability (acks):** `acks=all` (wait for replicas) is durable but slower; `acks=1` is faster but can lose data on leader failure.
- **At-least-once vs exactly-once:** exactly-once adds overhead/complexity; at-least-once + idempotent consumers is the common pragmatic choice.

- **Retention (storage) vs replay ability:** longer retention enables replay/new consumers but costs disk.

AWS Well-Architected Assessment

- **Operational Excellence:** managed MSK/Kinesis reduces broker ops; CloudWatch on consumer lag, throughput, under-replicated partitions; IaC.
 - **Security:** TLS + SASL/IAM auth; encryption at rest; ACLs per topic; VPC isolation; least-privilege.
 - **Reliability:** replication across AZs + ISR; leader failover; durable persisted offsets; no message loss with `acks=all`.
 - **Performance Efficiency:** partition-level parallelism; sequential I/O + batching; scale partitions/brokers to load.
 - **Cost Optimization:** right-size retention and partitions; tiered storage for older log segments; scale brokers to throughput.
 - **Sustainability:** efficient sequential I/O and batching; retention tuned to need (avoid storing unused data); autoscale to real throughput.
-

17. Leaderboard / Ranking System



Problem: Design a real-time leaderboard showing top players (and a player's rank) for a game with millions of players.

1. Functional Requirements

- **Update a player's score.**
- **Get top-K** players.
- **Get a player's rank** (and nearby players).
- **Time-based boards** (daily/weekly) (mention).
- *Out of scope:* anti-cheat, matchmaking.

2. Non-Functional Requirements

- **Low latency** reads/writes (real-time updates).
- **Scale:** millions of players; frequent score updates.
- **Accuracy:** ranks should be correct/consistent.
- **High availability.**

3. Core Entities

- **Player** (playerId , name), **Score** (playerId , score , updatedAt), **Leaderboard** (a ranked set, possibly per-time-window/region).

4. The API

```
POST /scores          → { playerId, score } (update)
GET  /leaderboard/top?k= → top-K players
GET  /leaderboard/rank/{playerId} → rank + surrounding players
```

5. High-Level Design

```
Client → Leaderboard Service → Redis Sorted Set (ZADD/ZREVRANGE/ZRANK)
                                   |
                                   ↳ Durable DB (source of truth for scores)
```

- **Redis Sorted Set (ZSET)** is the ideal primitive: it keeps members ordered by score with $O(\log N)$ inserts and $O(\log N + K)$ range reads.
- `ZADD board score player` → update score.
- `ZREVRANGE board 0 K-1` → top-K.
- `ZREVRANK board player` → a player's rank.
- Persist scores to a **durable DB** as the source of truth; Redis serves the fast ranking.

6. Potential Deep Dives

- **Why sorted sets:** they solve top-K and rank queries in log time — a relational `ORDER BY score + COUNT` for rank is $O(N)$ and far too slow at scale. This data-structure choice is the design.
- **Scaling beyond one node:** a huge leaderboard may exceed one Redis node.
- *Sharding:* partition players across shards, keep per-shard top-K, and merge for the global top-K. Global exact **rank** across shards is harder → approximate (bucketed) rank, or a two-level structure.
- *Replication* for read scaling/availability.
- **Time-based boards:** separate sorted sets per window (daily/weekly) with TTL; roll over at boundaries.
- **Consistency & durability:** Redis is fast but volatile → write scores to a durable store too; on Redis failure, rebuild the sorted set from the DB.
- **Hot updates:** extremely frequent updates for top players → batching/throttling; Redis handles high write rates well.
- **Rank of a specific player at massive scale:** exact rank is expensive when sharded → use **percentile/bucketed rank** approximations for "you're in the top 5%".

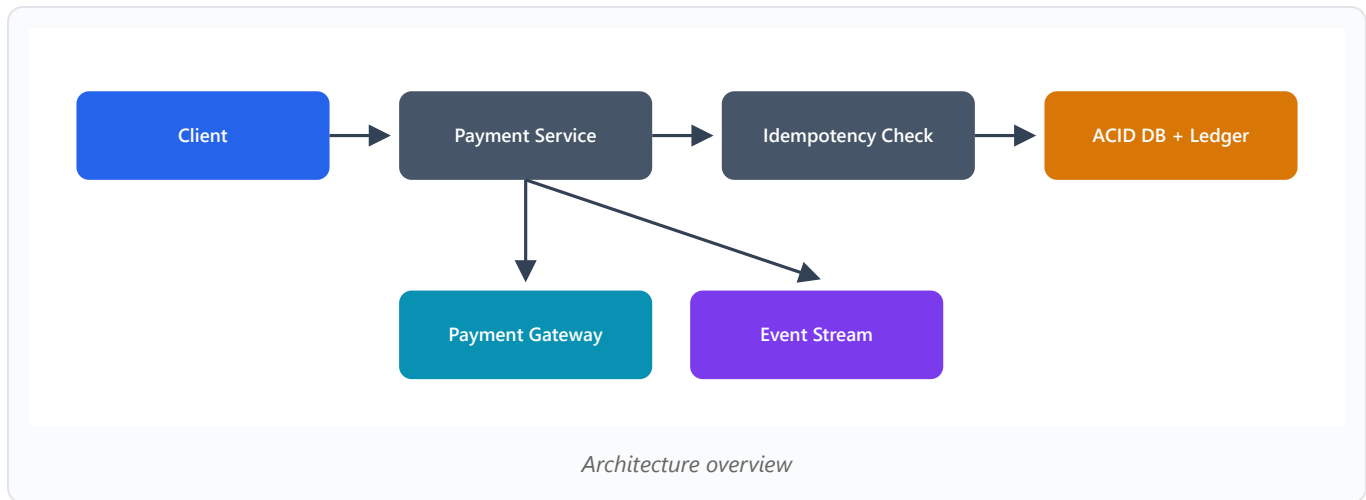
7. Trade-offs

- **Redis (in-memory) vs durable DB alone:** Redis gives the needed latency but is volatile → pair with a durable store (extra write). Correct trade for real-time ranking.
- **Exact vs approximate rank at scale:** exact global rank across shards is costly; approximate rank is fast and usually good enough.
- **Single node (simple, exact) vs sharded (scalable, approximate):** choose based on player count.

AWS Well-Architected Assessment

- **Operational Excellence:** ElastiCache (Redis) managed; CloudWatch on update/read latency; DB as recoverable source of truth; IaC.
 - **Security:** authN on score submission (prevent spoofing); server-side score validation (anti-cheat hook); TLS; least-privilege.
 - **Reliability:** Redis multi-AZ replicas; rebuild sorted set from durable DB after failure; no single point of data loss.
 - **Performance Efficiency:** sorted sets for $O(\log N)$ ops; sharding for scale; replicas for read throughput.
 - **Cost Optimization:** in-memory only for the ranking structure (compact); durable DB for cold storage; right-size Redis; TTL time-based boards.
 - **Sustainability:** efficient data structures avoid $O(N)$ scans; autoscale to player activity; expire old boards.
-

18. Payment System / Digital Wallet



Problem: Design a system to process payments and manage account balances correctly, with no lost or duplicated money.

1. Functional Requirements

- **Make a payment / transfer** between accounts.
- **Maintain balances** accurately.
- **Transaction history / ledger.**
- **Idempotent** payment requests (safe retries).
- *Out of scope:* fraud ML, currency conversion details.

2. Non-Functional Requirements

- **Strong consistency & correctness:** money must never be lost, created, or double-spent — the absolute priority.
- **Durability & auditability:** every transaction permanently recorded.
- **Exactly-once effect** (via idempotency).
- **Availability**, but **correctness > availability** here.
- **Security/compliance** (PCI-DSS).

3. Core Entities

- **Account** (`accountId`, `balance`), **Transaction** (`txnId`, `from`, `to`, `amount`, `status`, `idempotencyKey`), **Ledger Entry** (immutable double-entry records), **Payment** (external gateway interaction).

4. The API

```
POST /payments → { fromAccount, toAccount, amount, idempotencyKey } → { txnId, status }
GET /payments/{txnId}
GET /accounts/{id}/transactions
```

5. High-Level Design

```
Client → Payment Service → (idempotency check) → ACID DB (accounts + double-entry ledger)
                                     |
                                     ↳ External Payment Gateway (for card/bank rails)
                                     ↳ Event stream (async: notifications, analytics, reconciliation)
```

- **ACID transactions** update balances and write **ledger** entries atomically (debit one account, credit another — double-entry) so the books always balance.
- **Idempotency keys** ensure a retried request doesn't process twice.
- **Double-entry ledger** (immutable append-only records) is the auditable source of truth; balances can be derived/validated from it.

6. Potential Deep Dives

- **Idempotency (critical):** the client sends a unique `idempotencyKey`; the server records processed keys and returns the original result on retry → **exactly-once effect** despite network retries. Prevents double charges.
- **Consistency for balances:** use an **ACID database** with transactions (and appropriate isolation) so concurrent transfers can't overspend. For distributed accounts/services, avoid unsafe dual writes — use the **Saga pattern** with compensating actions, or a transactional outbox, for cross-service payments.
- **Double-entry ledger:** every transaction writes balanced debit/credit entries; the ledger is **append-only and immutable** (never update/delete) → full audit trail and easy reconciliation.
- **External gateway integration:** calling card networks/banks is asynchronous and can fail/time out → track payment state machine (pending→authorized→captured→settled/failed); reconcile with the provider; retry safely (idempotent).
- **Exactly-once with external systems:** combine idempotency keys, a state machine, and reconciliation jobs to detect/heal discrepancies.
- **Security/compliance:** never store raw card data (tokenize via PCI-compliant vault); encrypt everything; strict access controls and audit logs.

7. Trade-offs

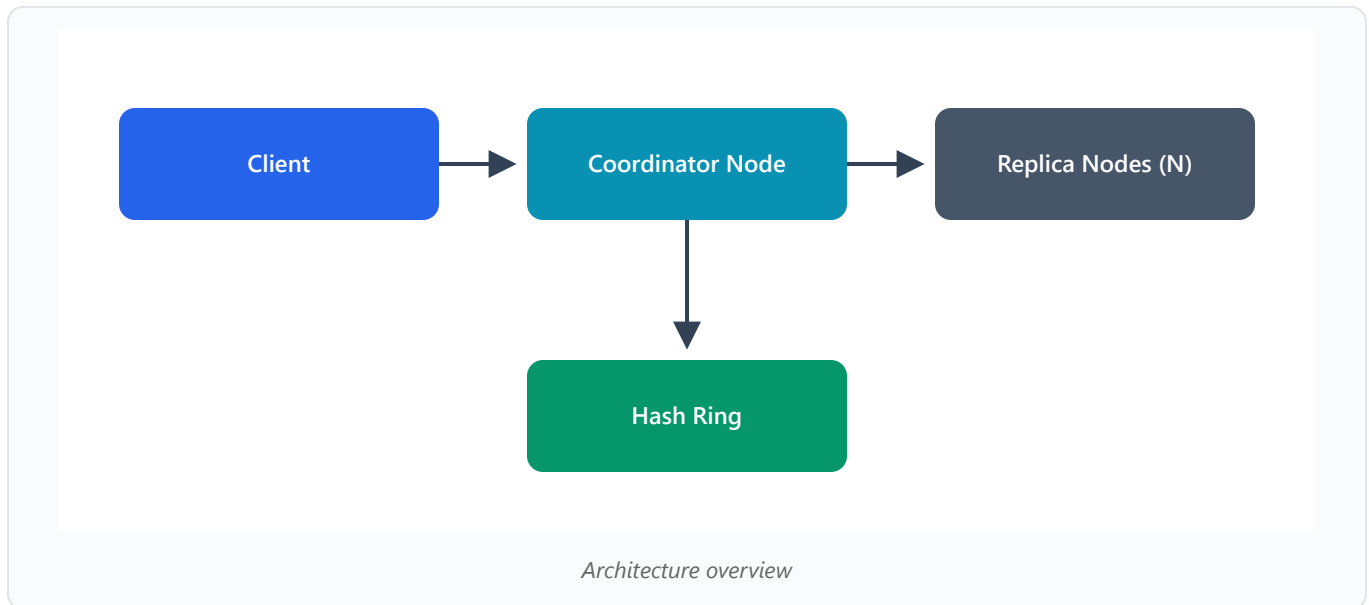
- **Consistency vs availability/scale:** payments choose **strong consistency** (ACID) over scale — correctness is non-negotiable. Scale reads (history) separately.
- **Sync vs async settlement:** authorize synchronously for UX; settle asynchronously (with reconciliation) since external rails are slow.
- **Saga vs 2PC for distributed payments:** 2PC's locking hurts availability; sagas (eventual consistency + compensation) are preferred, at the cost of more complex failure handling.

- **Ledger immutability (storage) vs simplicity:** an append-only ledger costs storage but is essential for auditability/correctness.

AWS Well-Architected Assessment

- **Operational Excellence:** ACID DB (Aurora); reconciliation jobs; detailed audit logging; CloudWatch alarms on failed/stuck payments; game days; IaC.
 - **Security:** PCI-DSS scope minimization (tokenization/vault); encryption at rest/in transit; strict least-privilege + separation of duties; immutable audit logs; secrets in Secrets Manager. **Highest-priority pillar here.**
 - **Reliability:** multi-AZ Aurora; idempotency prevents duplicates; sagas + reconciliation heal partial failures; durable ledger; defined RPO $\approx$ 0.
 - **Performance Efficiency:** strong-consistency core kept lean; async offload of non-critical work (notifications, analytics); scale the read/history path with replicas/cache.
 - **Cost Optimization:** reserve strong-consistency resources only for the money path; async/batch the rest; right-size; avoid over-scaling the low-QPS write core.
 - **Sustainability:** efficient transactional core (no wasteful polling); async batching; right-sized durable storage; archive old ledger data to cold tiers.
-

19. Key-Value Store (DynamoDB / Cassandra-like)



Problem: Design a distributed, highly available key-value store that scales horizontally.

1. Functional Requirements

- **put(key, value), get(key), delete(key).**
- **Horizontal scalability** (add nodes for capacity/throughput).
- **High availability** (survive node/AZ failures).
- **Tunable consistency.**

2. Non-Functional Requirements

- **High availability** (often AP — always writable).
- **Scalability:** near-linear with nodes.
- **Low latency** at scale.
- **Durability:** replicated, no data loss.
- **Partition tolerance** (it's distributed).

3. Core Entities

- **Key/Value pair**, **Node** (owns partitions), **Partition** (key range/hash bucket), **Replica** (copy of a partition), **Ring** (consistent-hash topology), **Version** (vector clock / timestamp).

4. The API

```
put(key, value, [consistencyLevel])
get(key, [consistencyLevel]) → value(s)
```

```
delete(key)
```

5. High-Level Design

Client → (coordinator node) → consistent hash(key) → N replica nodes
write to W replicas / read from R replicas (quorum)

- **Partitioning via consistent hashing** distributes keys across nodes with minimal remapping when nodes change (virtual nodes for balance).
- **Replication:** each key is stored on **N** nodes (the next N on the ring).
- **Tunable consistency via quorums:** a write succeeds on **W** replicas, a read queries **R** replicas; if **W + R > N**, reads see the latest write (strong-ish). Tune W/R for latency vs consistency.
- **Leaderless** design (Dynamo-style): any replica can take writes → high availability.

6. Potential Deep Dives

- **Consistent hashing + virtual nodes:** even key distribution and smooth rebalancing when nodes join/leave (only $\sim 1/N$ keys move). Foundational for horizontal scale.
- **Replication & quorums (N/W/R):** $W + R > N$ → strong consistency; $W + R \leq N$ → faster but eventually consistent. E.g., $N=3, W=2, R=2$ balances both. Availability: writes succeed as long as W replicas are up.
- **Conflict resolution:** with leaderless writes, concurrent updates conflict → detect with **vector clocks** (or version numbers); resolve by last-write-wins or application merge (e.g., merge shopping carts). Read-repair and anti-entropy (Merkle trees) reconcile divergent replicas.
- **Handling failures: hinted handoff** — if a replica is down, another node temporarily stores the write and forwards it later, preserving availability and durability.
- **Storage engine:** LSM-tree (memtable + SSTables + commit log) for high write throughput (Cassandra) — sequential writes, background compaction.
- **Gossip protocol:** nodes share membership/health info peer-to-peer (no central coordinator) → decentralized, no SPOF.

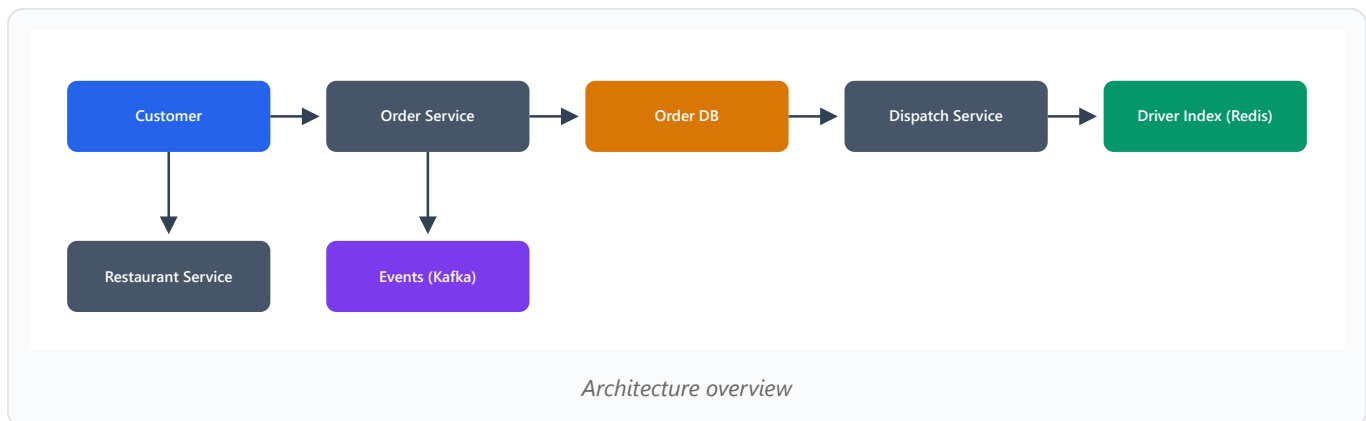
7. Trade-offs

- **Consistency vs availability/latency (CAP/PACELC):** tunable via N/W/R — the core trade. AP by default (available, eventually consistent); can lean CP with higher quorums.
- **Leaderless (highly available, conflict resolution needed) vs leader-based (simpler consistency, failover downtime).**
- **LWW (simple, may lose writes) vs vector clocks (accurate, complex).**
- **Read-repair/anti-entropy overhead** vs consistency — background reconciliation costs resources.

AWS Well-Architected Assessment

- **Operational Excellence:** DynamoDB (fully managed) removes most ops; CloudWatch on latency, throttling, replication; IaC. (Cassandra self-managed = more ops via Keyspaces alternative.)
 - **Security:** encryption at rest/in transit; fine-grained IAM; VPC endpoints; least-privilege.
 - **Reliability:** replication across AZs/regions (global tables); leaderless design + hinted handoff = high availability; no SPOF (gossip, no central master); tunable durability.
 - **Performance Efficiency:** consistent hashing for even load; LSM for write-heavy workloads; quorum tuning for latency; single-digit-ms reads.
 - **Cost Optimization:** on-demand/auto-scaling capacity; tiered storage; tune replication factor to durability needs; avoid over-replication.
 - **Sustainability:** near-linear scaling avoids over-provisioning; efficient LSM writes; autoscale to demand; multi-tenant managed service improves utilization.
-

20. DoorDash / Food Delivery



Problem: Design a food-delivery platform connecting customers, restaurants, and delivery drivers.

1. Functional Requirements

- **Browse restaurants/menus** near the customer.
- **Place an order.**
- **Restaurant accepts & prepares.**
- **Match a driver** to pick up and deliver.
- **Real-time tracking** of the order/driver.
- *Out of scope:* payments (Q18), reviews.

2. Non-Functional Requirements

- **Three-sided marketplace** (customers, restaurants, drivers) — all real-time.
- **Geospatial:** nearby restaurants and driver matching.
- **Low latency** browsing + real-time tracking.
- **Reliability:** orders must not be lost; consistent order state.
- **Scale:** millions of orders/day, spiky (meal times).

3. Core Entities

- **Customer, Restaurant** (id , location , menu , status), **Order** (orderId , items , status , customerId , restaurantId , driverId), **Driver** (id , location , status), **Delivery** (pickup→dropoff).

4. The API

GET /restaurants?near=lat,lng	→ nearby restaurants
POST /orders	→ { restaurantId, items } → { orderId }
PATCH /orders/{id}/status	→ restaurant/driver update state

POST /drivers/location	→ driver location pings
WebSocket	→ live order/driver tracking

5. High-Level Design

Browse: Client → Restaurant Service → Geospatial index (nearby) + Menu DB (+ cache/CDN)
Order: Client → Order Service → Order DB (state machine) → notify restaurant
Dispatch: Order → Dispatch/Matching Service → Driver location index → assign driver
Track: All parties ⇌ WebSocket gateway ⇌ live location/status updates

- **Order as a state machine:** placed → accepted → preparing → ready → picked-up → delivered. The **Order Service** owns this lifecycle reliably.
- **Geospatial** powers both nearby-restaurant browsing and driver matching (like Uber, Q10).
- **Real-time tracking** over WebSockets to customer, restaurant, and driver.
- **Async events** (Kafka) coordinate the three parties and downstream work (notifications, ETAs, analytics).

6. Potential Deep Dives

- **Driver matching & dispatch:** find available drivers near the restaurant (geospatial index), optimize assignment (proximity, current load, batching multiple orders on one route). This is a real-time optimization problem; start with nearest-available, evolve to batched routing.
- **Order state consistency:** the order lifecycle must be reliable and consistent across three parties → an authoritative Order Service with a clear state machine; use events to propagate transitions; idempotent updates (retries mustn't skip/duplicate states).
- **Geospatial at scale:** geohash/quadtree for restaurants (mostly static) and drivers (high-frequency updates → in-memory like Uber). Shard by region/city (natural geo-partitioning).
- **ETA prediction:** combine prep time + driver travel time (traffic) → ML/heuristics; update in real time.
- **Spiky load:** lunch/dinner peaks → autoscale; queue order events to absorb bursts; cache menus/restaurant data (CDN) since browsing dominates reads.
- **Real-time connections:** WebSocket gateway + session registry (like chat, Q5).

7. Trade-offs

- **Strong consistency for order state vs eventual for tracking:** order lifecycle needs consistency (don't lose/duplicate orders); live location can be eventually consistent/best-effort.
- **Matching optimality vs latency:** globally optimal dispatch (batching, routing) is expensive; a fast nearest-driver match is responsive — evolve from simple to optimized.
- **Monolith vs microservices:** three-sided domain naturally splits (restaurant, order, dispatch, driver services) — but adds distributed complexity; justified at this scale.
- **Caching menus (stale risk) vs freshness:** cache heavily for browse performance, invalidate on menu/price changes.

AWS Well-Architected Assessment

- **Operational Excellence:** microservices with clear ownership; Kafka/MSK for event coordination; CloudWatch on order latency, match time, delivery SLAs; IaC; regional deploys.
 - **Security:** authN/authZ per party (customer/restaurant/driver); protect location/PII; least-privilege; secure payment handoff (Q18); audit order changes.
 - **Reliability:** durable Order DB + event log; idempotent state transitions; multi-AZ; regional isolation limits outage blast radius; graceful degradation (stale ETAs if traffic stream lags).
 - **Performance Efficiency:** geospatial indexes; in-memory driver locations; CDN/cache for menus; async events for coordination; WebSockets for tracking.
 - **Cost Optimization:** autoscale to meal-time peaks then down; cache-heavy browse path; in-memory only for ephemeral driver location; Spot for batch/analytics.
 - **Sustainability:** geo-partitioning avoids global scans; batched deliveries cut driver miles; autoscaling to demand peaks; caching reduces recompute; efficient location pings.
-

Closing note

Every design above used the same discipline: **clarify requirements** → **size the system** → **model the data** → **define the API** → **design the architecture** → **deep-dive the hard parts** → **own the trade-offs** → **validate against the AWS Well-Architected pillars**. In an interview, this *process* — not any single "right" diagram — is what demonstrates senior-level system design skill. Adapt the depth to the time available, and always tie decisions back to the stated requirements.